

Memory Barriers: a Hardware View for Software Hackers

Paul E. McKenney
Linux Technology Center
IBM Beaverton
paulmck@us.ibm.com

April 5, 2009

So what possessed CPU designers to cause them to inflict memory barriers on poor unsuspecting SMP software designers?

In short, because reordering memory references allows much better performance, and so memory barriers are needed to force ordering in things like synchronization primitives whose correct operation depends on ordered memory references.

Getting a more detailed answer to this question requires a good understanding of how CPU caches work, and especially what is required to make caches really work well. The following sections:

1. present the structure of a cache,
2. describe how cache-coherency protocols ensure that CPUs agree on the value of each location in memory, and, finally,
3. outline how store buffers and invalidate queues help caches and cache-coherency protocols achieve high performance.

We will see that memory barriers are a necessary evil that is required to enable good performance and scalability, an evil that stems from the fact that CPUs are orders of magnitude faster than are both the interconnects between them and the memory they are attempting to access.

1 Cache Structure

Modern CPUs are much faster than are modern memory systems. A 2006 CPU might be capable of execut-

ing ten instructions per nanosecond, but will require many tens of nanoseconds to fetch a data item from main memory. This disparity in speed — more than two orders of magnitude — has resulted in the multi-megabyte caches found on modern CPUs. These caches are associated with the CPUs as shown in Figure 1, and can typically be accessed in a few cycles.¹

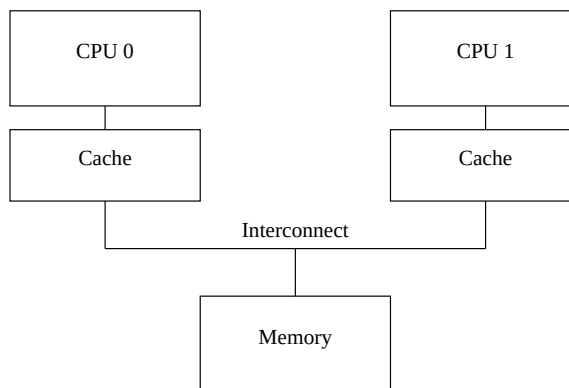


Figure 1: Modern Computer System Cache Structure

Data flows among the CPUs' caches and memory in fixed-length blocks called “cache lines”, which are normally a power of two in size, ranging from 16 to 256 bytes. When a given data item is first accessed by

¹It is standard practice to use multiple levels of cache, with a small level-one cache close to the CPU with single-cycle access time, and a larger level-two cache with a longer access time, perhaps roughly ten clock cycles. Higher-performance CPUs often have three or even four levels of cache.

内存屏障：软件黑客的硬件视角

保罗·E·麦肯尼 IBM Linux
技术中心 贝夫顿实验室
paulmck@us.ibm.com

2009年4月5日

是什么让CPU设计者做出这样的决定，迫使他们将内存屏障强加给无辜且不知情的SMP软件设计者？

简而言之，由于重新排序内存引用可以显著提升性能，因此在诸如正确运行依赖于有序内存引用的同步原语等场景中，需要使用内存屏障来强制执行顺序。

要更详细地回答这个问题，需要深入了解CPU缓存的工作原理，特别是使缓存真正高效运行所需的条件。以下各节：

呈现缓存的结构，

2. 描述缓存一致性协议如何确保各个CPU对内存中每个位置的值达成一致，并最终实现这一点。

3. 描述存储缓冲区和无效队列如何帮助缓存及缓存一致性协议实现高性能。

内存屏障是一种必要的机制，虽然它带来了复杂性，但却是实现良好性能和可扩展性的前提。这种复杂性源于CPU的运行速度远远超过其间的互连以及它们试图访问的内存的速度。

1 Cache Structure

现代CPU的运行速度远超现代内存系统。一个2006年的CPU可能具备每秒执行数十亿次操作的能力，而内存系统的性能则远远落后。

在一个纳秒内可以执行十条指令，但需要数十纳秒才能从主内存中获取一个数据项。这种速度上的差异——相差两个数量级以上——导致了现代CPU上出现的多兆字节缓存。这些缓存与CPU相关联，如图1所示，并且通常可以在几个时钟周期内访问到。

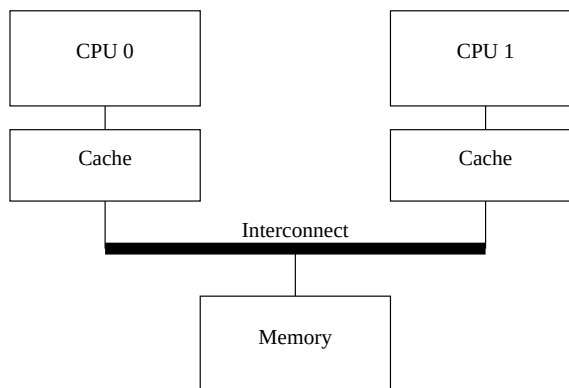


图1：现代计算机系统缓存结构

数据在CPU缓存和内存之间以固定长度的块（称为“缓存行”）进行传输，这些块通常为2的幂大小，范围从16字节到256字节。当某个数据项首次被访问时

¹It is standard practice to use multiple levels of cache, with a small level-one cache close to the CPU with single-cycle access time, and a larger level-two cache with a longer access time, perhaps roughly ten clock cycles. Higher-performance CPUs often have three or even four levels of cache.

a given CPU, it will be absent from that CPU’s cache, meaning that a “cache miss” (or, more specifically, a “startup” or “warmup” cache miss) has occurred. The cache miss means that the CPU will have to wait (or be “stalled”) for hundreds of cycles while the item is fetched from memory. However, the item will be loaded into that CPU’s cache, so that subsequent accesses will find it in the cache and therefore run at full speed.

After some time, the CPU’s cache will fill, and subsequent misses will likely need to eject an item from the cache in order to make room for the newly fetched item. Such a cache miss is termed a “capacity miss”, because it is caused by the cache’s limited capacity. However, most caches can be forced to eject an old item to make room for a new item even when they are not yet full. This is due to the fact that large caches are implemented as hardware hash tables with fixed-size hash buckets (or “sets”, as CPU designers call them) and no chaining, as shown in Figure 2.

This cache has sixteen “sets” and two “ways” for a total of 32 “lines”, each entry containing a single 256-byte “cache line”, which is a 256-byte-aligned block of memory. This cache line size is a little on the large size, but makes the hexadecimal arithmetic much simpler. In hardware parlance, this is a two-way set-associative cache, and is analogous to a software hash table with sixteen buckets, where each bucket’s hash chain is limited to at most two elements. The size (32 cache lines in this case) and the associativity (two in this case) are collectively called the cache’s “geometry”. Since this cache is implemented in hardware, the hash function is extremely simple: extract four bits from the memory address.

In Figure 2, each box corresponds to a cache entry, which can contain a 256-byte cache line. However, a cache entry can be empty, as indicated by the empty boxes in the figure. The rest of the boxes are flagged with the memory address of the cache line that they contain. Since the cache lines must be 256-byte aligned, the low eight bits of each address are zero, and the choice of hardware hash function means that the next-higher four bits match the hash line number.

The situation depicted in the figure might arise if the program’s code were located at address

	Way 0	Way 1
0x0	0x12345000	
0x1	0x12345100	
0x2	0x12345200	
0x3	0x12345300	
0x4	0x12345400	
0x5	0x12345500	
0x6	0x12345600	
0x7	0x12345700	
0x8	0x12345800	
0x9	0x12345900	
0xA	0x12345A00	
0xB	0x12345B00	
0xC	0x12345C00	
0xD	0x12345D00	
0xE	0x12345E00	0x43210E00
0xF		

Figure 2: CPU Cache Structure

0x43210E00 through 0x43210EFF, and this program accessed data sequentially from 0x12345000 through 0x12345EFF. Suppose that the program were now to access location 0x12345F00. This location hashes to line 0xF, and both ways of this line are empty, so the corresponding 256-byte line can be accommodated. If the program were to access location 0x1233000, which hashes to line 0x0, the corresponding 256-byte cache line can be accommodated in way 1. However, if the program were to access location 0x1233E00, which hashes to line 0xE, one of the existing lines must be ejected from the cache to make room for the new cache line. If this ejected line were accessed later, a cache miss would result. Such a cache miss is termed an “associativity miss”.

Thus far, we have been considering only cases where a CPU reads a data item. What happens when it does a write? Because it is important that all CPUs agree on the value of a given data item, before a given CPU writes to that data item, it must first cause it to be removed, or “invalidated”, from other CPUs’ caches. Once this invalidation has completed, the CPU may safely modify the data item. If the data item was present in this CPU’s cache, but was read-only, this process is termed a “write miss”. Once a given CPU has completed invalidating a given data item from other CPUs’ caches, that CPU may repeat-

对于给定的CPU而言，该数据将不会存在于其缓存中，这意味着发生了“缓存未命中”（更具体地说是“启动”或“预热”阶段的缓存未命中）。缓存未命中意味着CPU必须等待数百个时钟周期，直到从内存中获取该项数据。然而，该项数据会被加载到该CPU的缓存中，因此后续访问将直接在缓存中找到该项数据，并以全速运行。

经过一段时间，CPU的缓存将被填满，随后的缺失（缺页异常）很可能需要从缓存中移除一个项目以腾出空间给新获取的项目。这种缓存缺失被称为“容量缺失”，因为它是由缓存有限的容量引起的。然而，大多数缓存即使在未完全填满的情况下也可以被迫移除旧项来为新项腾出空间。这是由于大容量缓存通常作为硬件哈希表实现，其中包含固定大小的哈希桶（或称为“集合”，如CPU设计者所称），并且不使用链接机制，如图2所示。

此缓存包含十六个“组”和两个“路”，总共为32条“缓存行”，每个条目包含一个单个的256字节“缓存行”，即内存中的一个256字节对齐的块。这种缓存行大小略大一些，但使十六进制运算变得简单得多。在硬件术语中，这是一个两路组相联缓存，并且类似于具有十六个桶的软件哈希表，其中每个桶的哈希链最多包含两个元素。该缓存的大小（本例中有32条缓存行）和关联性（本例中为两路）统称为缓存的“几何”。由于此缓存是在硬件中实现的，因此哈希函数非常简单：从内存地址中提取四个比特。

如图2所示，每个方框对应一个缓存条目，可以包含一条256字节的缓存行。然而，缓存条目也可以为空，如图中空方框所示。其余的方框则标记有其所含缓存行的内存地址。由于缓存行必须是256字节对齐的，因此每个地址的低8位为0，并且硬件哈希函数的选择意味着接下来的4位与哈希线号匹配。

该图所示的情况可能会出现在程序代码位于地址的情况下。

	Way 0	Way 1
0x0	0x12345000	
0x1	0x12345100	
0x2	0x12345200	
0x3	0x12345300	
0x4	0x12345400	
0x5	0x12345500	
0x6	0x12345600	
0x7	0x12345700	
0x8	0x12345800	
0x9	0x12345900	
0xA	0x12345A00	
0xB	0x12345B00	
0xC	0x12345C00	
0xD	0x12345D00	
0xE	0x12345E00	0x43210E00
0xF		

图2：CPU缓存结构

0x43210E00 至 0x43210EFF，该程序依次访问了从 0x12345000 到 0x12345EFF 的数据。假设程序现在要访问位置 0x12345F00。此位置哈希到缓存行 0xF，并且该行的两个路均为空，因此可以容纳对应的 256 字节缓存行。如果程序访问位置 0x1233000（该位置哈希到缓存行 0x0），则可以在路 1 中容纳对应的 256 字节缓存行。然而，若程序访问位置 0x1233E00（该位置哈希到缓存行 0xE），必须将现有的一条缓存行从缓存中移除以腾出空间给新的缓存行。如果后来再次访问被移除的这条缓存行，则会导致缓存未命中。这种类型的缓存未命中被称为“关联性未命中”。

到目前为止，我们只考虑了CPU读取数据项的情况。当CPU执行写操作时会发生什么？由于所有CPU必须对给定数据项的值达成一致，因此在某个CPU向该数据项写入之前，它必须首先导致其他CPU缓存中的该数据项被移除或“失效”。一旦完成此失效过程，该CPU可以安全地修改数据项。如果该数据项存在于当前CPU的缓存中但为只读，则这一过程称为“写缺失”。在某个CPU完成了将给定数据项从其他CPU缓存中失效后，该CPU可以重复上述步骤。

edly write (and read) that data item.

Later, if one of the other CPUs attempts to access the data item, it will incur a cache miss, this time because the first CPU invalidated the item in order to write to it. This type of cache miss is termed a “communication miss”, since it is usually due to several CPUs using the data items to communicate (for example, a lock is a data item that is used to communicate among CPUs using a mutual-exclusion algorithm).

Clearly, much care must be taken to ensure that all CPUs maintain a coherent view of the data. With all this fetching, invalidating, and writing, it is easy to imagine data being lost or (perhaps worse) different CPUs having conflicting values for the same data item in their respective caches. These problems are prevented by “cache-coherency protocols”, described in the next section.

2 Cache-Coherence Protocols

Cache-coherency protocols manage cache-line states so as to prevent inconsistent or lost data. These protocols can be quite complex, with many tens of states,² but for our purposes we need only concern ourselves with the four-state MESI cache-coherence protocol.

2.1 MESI States

MESI stands for “modified”, “exclusive”, “shared”, and “invalid”, the four states a given cache line can take on using this protocol. Caches using this protocol therefore maintain a two-bit state “tag” on each cache line in addition to that line’s physical address and data.

A line in the “modified” state has been subject to a recent memory store from the corresponding CPU, and the corresponding memory is guaranteed not to appear in any other CPU’s cache. Cache lines in the “modified” state can thus be said to be “owned” by

the CPU. Because this cache holds the only up-to-date copy of the data, this cache is ultimately responsible for either writing it back to memory or handing it off to some other cache, and must do so before reusing this line to hold other data.

The “exclusive” state is very similar to the “modified” state, the single exception being that the cache line has not yet been modified by the corresponding CPU, which in turn means that the copy of the cache line’s data that resides in memory is up-to-date. However, since the CPU can store to this line at any time, without consulting other CPUs, a line in the “exclusive” state can still be said to be owned by the corresponding CPU. That said, because the corresponding value in memory is up to date, this cache can discard this data without writing it back to memory or handing it off to some other CPU.

A line in the “shared” state might be replicated in at least one other CPU’s cache, so that this CPU is not permitted to store to the line without first consulting with other CPUs. As with the “exclusive” state, because the corresponding value in memory is up to date, this cache can discard this data without writing it back to memory or handing it off to some other CPU.

A line in the “invalid” state is empty, in other words, it holds no data. When new data enters the cache, it is placed into a cache line that was in the “invalid” state if possible. This approach is preferred because replacing a line in any other state could result in an expensive cache miss should the replaced line be referenced in the future.

Since all CPUs must maintain a coherent view of the data carried in the cache lines, the cache-coherence protocol provides messages that coordinate the movement of cache lines through the system.

2.2 MESI Protocol Messages

Many of the transitions described in the previous section require communication among the CPUs. If the CPUs are on a single shared bus, the following messages suffice:

Read: The “read” message contains the physical address of the cache line to be read.

²See Culler et al. [3] pages 670 and 671 for the nine-state and 26-state diagrams for SGI Origin2000 and Sequent (now IBM) NUMA-Q, respectively. Both diagrams are significantly simpler than real life.

正确写入（并读取）该数据项。

后来，如果其他某个CPU尝试访问该数据项时，将会发生缓存未命中，这次是因为第一个CPU为了写入数据而使该项失效。这种类型的缓存未命中被称为“通信未命中”，因为通常这是由于多个CPU使用数据项进行通信所导致的（例如，一个锁就是一个用于在采用互斥算法的CPU之间进行通信的数据项）。

显然，必须采取周密的措施确保所有CPU对数据保持一致的看法。在不断获取、失效和写入的过程中，很容易想象数据可能会丢失，或者（也许更糟糕的是）不同的CPU在其各自的缓存中为同一数据项持有冲突的值。这些问题通过“缓存一致性协议”来预防，这些协议将在下一节中描述。

2 Cache-Coherence Protocols

缓存一致性协议管理缓存行的状态，以防止数据不一致或丢失。这些协议可能非常复杂，包含数十种状态，但对于我们而言，只需关注四态的MESI缓存一致性协议即可。

2.1 MESI States

MESI 分别代表“修改”、“独占”、“共享”和“无效”，这是使用该协议的缓存行可以处于的四种状态。采用此协议的缓存因此会在每条缓存行上除了物理地址和数据之外，还维护一个两位的状态标记（标签）。

处于“修改”状态的缓存行最近曾被相应的CPU进行了内存写操作，且对应的内存数据保证不会出现在其他任何CPU的缓存中。因此，可以将处于“修改”状态的缓存行说成是“被相应CPU所独占”的。

由于这个缓存中持有数据的唯一最新副本，因此该缓存最终负责将数据写回内存或将数据传递给其他缓存，并且必须在重用此缓存行来存储其他数据之前完成上述操作。

“独占”状态与“修改”状态非常相似，唯一的区别在于缓存行尚未被相应的CPU修改过，这意味着内存中该缓存行数据的副本是最新的。然而，由于CPU可以在任何时候存储到这条缓存行而无需与其他CPU进行通信，“独占”状态下的缓存行仍然可以被认为是相应CPU拥有的。不过，因为内存中的值是最新的，所以这个缓存可以丢弃这些数据而不必将其写回内存或将数据传递给其他CPU。

处于“共享”状态的一行可能还被至少一个其他CPU的缓存复制，因此该CPU在对该行进行写操作之前必须先与其他CPU进行协商。与“独占”状态类似，由于内存中的相应值是最新的，此缓存可以丢弃这些数据而无需将其回写到内存或将数据传递给其他CPU。

处于“无效”状态的缓存行是空的，换句话说，它不包含任何数据。当新数据进入缓存时，如果可能的话，会将其放置到一个处于“无效”状态的缓存行中。这种做法被优先采用是因为替换其他任何状态下的缓存行可能会导致将来引用该缓存行时发生代价高昂的缓存未命中。

由于所有CPU都必须维护缓存行中数据的一致视图，因此缓存一致性协议提供了协调缓存行在系统中移动的消息。

2.2 MESI Protocol Messages

许多前一节中描述的转换需要在CPU之间进行通信。如果这些CPU位于同一共享总线上，则以下消息就足够了：

Read: “读取”消息包含了要读取的缓存行的物理地址。

²See Culler et al. [3] pages 670 and 671 for the nine-state and 26-state diagrams for SGI Origin2000 and Sequent (now IBM) NUMA-Q, respectively. Both diagrams are significantly simpler than real life.

Read Response: The “read response” message contains the data requested by an earlier “read” message. This “read response” message might be supplied either by memory or by one of the other caches. For example, if one of the caches has the desired data in “modified” state, that cache must supply the “read response” message.

Invalidate: The “invalidate” message contains the physical address of the cache line to be invalidated. All other caches must remove the corresponding data from their caches and respond.

Invalidate Acknowledge: A CPU receiving an “invalidate” message must respond with an “invalidate acknowledge” message after removing the specified data from its cache.

Read Invalidate: The “read invalidate” message contains the physical address of the cache line to be read, while at the same time directing other caches to remove the data. Hence, it is a combination of a “read” and an “invalidate”, as indicated by its name. A “read invalidate” message requires both a “read response” and a set of “invalidate acknowledge” messages in reply.

Writeback: The “writeback” message contains both the address and the data to be written back to memory (and perhaps “snooped” into other CPUs’ caches along the way). This message permits caches to eject lines in the “modified” state as needed to make room for other data.

Interestingly enough, a shared-memory multiprocessor system really is a message-passing computer under the covers. This means that clusters of SMP machines that use distributed shared memory are using message passing to implement shared memory at two different levels of the system architecture.

Quick Quiz 1: What happens if two CPUs attempt to invalidate the same cache line concurrently? □

Quick Quiz 2: When an “invalidate” message appears in a large multiprocessor, every CPU must give an “invalidate acknowledge” response. Wouldn’t the resulting “storm” of “invalidate acknowledge” responses totally saturate the system bus? □

Quick Quiz 3: If SMP machines are really using message passing anyway, why bother with SMP at all? □

2.3 MESI State Diagram

A given cache line’s state changes as protocol messages are sent and received, as shown in Figure 3.

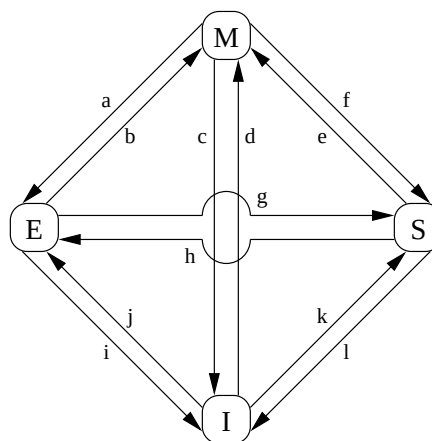


Figure 3: MESI Cache-Coherency State Diagram

The transition arcs in this figure are as follows:

Transition (a): A cache line is written back to memory, but the CPU retains it in its cache and further retains the right to modify it. This transition requires a “writeback” message.

Transition (b): The CPU writes to the cache line that it already had exclusive access to. This transition does not require any messages to be sent or received.

Transition (c): The CPU receives a “read invalidate” message for a cache line that it has modified. The CPU must invalidate its local copy, then respond with both a “read response” and an “invalidate acknowledge” message, both sending the data to the requesting CPU and indicating that it no longer has a local copy.

Read Response: “读响应”消息包含了一条早期“读”消息请求的数据。这条“读响应”消息可能是由内存提供，也可能是由其他缓存之一提供的。例如，如果某个缓存在“修改”状态中有所需数据，则该缓存必须提供“读响应”消息。

Invalidate: “无效”消息包含要被标记为无效的缓存行的物理地址。所有其他缓存必须从其缓存中移除相应数据并作出响应。

Invalidate Acknowledge: 接收“无效”消息的CPU必须在从其缓存中移除指定数据后，回复一个“无效确认”消息。

Read Invalidate: “读无效”消息包含了要读取的缓存行的物理地址，同时指示其他缓存移除数据。因此，它结合了“读取”和“无效”的功能，正如其名称所示。“读无效”消息需要一个“读响应”以及一系列“无效确认”消息作为回复。

Writeback: “写回”消息包含要写回到内存中的地址和数据（并且可能在途中被其他CPU的缓存监听到）。此消息允许缓存在需要时将处于“修改”状态的行逐出，以腾出空间供其他数据使用。

有趣的是，一个共享内存的对称多处理器系统实际上是一种基于消息传递的计算机。这意味着使用分布式共享内存的SMP机器集群在系统的两个不同层次架构上通过消息传递来实现共享内存。

Quick Quiz 1: 两个CPU同时尝试无效化同一缓存行会发生什么？

Quick Quiz 2: 当一个“无效”消息出现在大型多处理器系统中时，每个CPU都必须给出一个“无效确认”响应。这样的“无效确认”响应风暴会不会完全饱和系统总线？

Quick Quiz 3: 如果SMP机器实际上已经在使用消息传递机制，那么为何还要使用SMP呢？

2.3 MESI State Diagram

给定的缓存行的状态随着协议消息的发送和接收而发生变化，如图3所示。

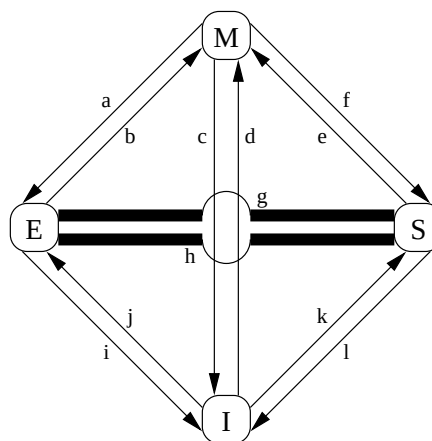


图3: MESI缓存一致性状态图

该图中的转换弧如下：

Transition (a): 缓存行被写回内存，但CPU仍然将其保留在缓存中，并保留进一步修改的权利。这一过渡需要发送一个“写回”消息。

Transition (b): CPU将其已获得独占访问权的缓存行进行写入。此转换无需发送或接收任何消息。

Transition (c): CPU收到一个针对已被修改的缓存行的“读无效”消息。CPU必须使其本地副本失效，然后回应一条包含数据并发送给请求方的“读响应”消息以及一条表示不再持有该缓存行本地副本的“无效确认”消息。

Transition (d): The CPU does an atomic read-modify-write operation on a data item that was not present in its cache. It transmits a “read invalidate”, receiving the data via a “read response”. The CPU can complete the transition once it has also received a full set of “invalidate acknowledge” responses.

Transition (e): The CPU does an atomic read-modify-write operation on a data item that was previously read-only in its cache. It must transmit “invalidate” messages, and must wait for a full set of “invalidate acknowledge” responses before completing the transition.

Transition (f): Some other CPU reads the cache line, and it is supplied from this CPU’s cache, which retains a read-only copy. This transition is initiated by the reception of a “read” message, and this CPU responds with a “read response” message containing the requested data.

Transition (g): Some other CPU reads a data item in this cache line, and it is supplied either from this CPU’s cache or from memory. In either case, this CPU retains a read-only copy. This transition is initiated by the reception of a “read” message, and this CPU responds with a “read response” message containing the requested data.

Transition (h): This CPU realizes that it will soon need to write to some data item in this cache line, and thus transmits an “invalidate” message. The CPU cannot complete the transition until it receives a full set of “invalidate acknowledge” responses. Alternatively, all other CPUs eject this cache line from their caches via “writeback” messages (presumably to make room for other cache lines), so that this CPU is the last CPU caching it.

Transition (i): Some other CPU does an atomic read-modify-write operation on a data item in a cache line held only in this CPU’s cache, so this CPU invalidates it from its cache. This transition is initiated by the reception of a “read invalidate” message, and this CPU responds with

both a “read response” and an “invalidate acknowledge” message.

Transition (j): This CPU does a store to a data item in a cache line that was not in its cache, and thus transmits a “read invalidate” message. The CPU cannot complete the transition until it receives the “read response” and a full set of “invalidate acknowledge” messages. The cache line will presumably transition to “modified” state via transition (b) as soon as the actual store completes.

Transition (k): This CPU loads a data item in a cache line that was not in its cache. The CPU transmits a “read” message, and completes the transition upon receiving the corresponding “read response”.

Transition (l): Some other CPU does a store to a data item in this cache line, but holds this cache line in read-only state due to its being held in other CPUs’ caches (such as the current CPU’s cache). This transition is initiated by the reception of an “invalidate” message, and this CPU responds with an “invalidate acknowledge” message.

Quick Quiz 4: How does the hardware handle the delayed transitions described above? ☐

2.4 MESI Protocol Example

Let’s now look at this from the perspective of a cache line’s worth of data, initially residing in memory at address 0, as it travels through the various single-line direct-mapped caches in a four-CPU system. Table 1 shows this flow of data, with the first column showing the sequence of operations, the second the CPU performing the operation, the third the operation being performed, the next four the state of each CPU’s cache line (memory address followed by MESI state), and the final two columns whether the corresponding memory contents are up to date (“V”) or not (“I”).

Initially, the CPU cache lines in which the data would reside are in the “invalid” state, and the data is valid in memory. When CPU 0 loads the data at

Transition (d): CPU对缓存中不存在的数据项执行原子读取-修改-写入操作。它发送“读无效”请求，并通过“读响应”接收数据。一旦收到完整的“无效确认”响应集，CPU即可完成该转换过程。

Transition (e): CPU 对之前只读缓存中的一个数据项执行原子的读取-修改-写入操作。它必须传输“无效”消息，并且在完成转换前必须等待完整的一组“无效确认”响应。

Transition (f): 其他CPU读取了缓存行，该缓存行来源于此CPU的缓存，其中保留了一份只读副本。这一状态转换由接收“读”消息触发，随后此CPU用包含请求数据的“读响应”消息进行回复。

Transition (g): 其他CPU读取了该缓存行中的一个数据项，这个数据要么来自此CPU的缓存，要么来自内存。无论哪种情况，此CPU都会保留一份只读副本。这一转换由接收“读”消息触发，并且此CPU会用包含请求数据的“读响应”消息进行回应。

Transition (h): 该CPU意识到它即将需要向此缓存行中的某个数据项写入数据，因此发送了一个“无效”消息。该CPU必须等到收到所有其他CPU的“无效确认”响应后才能完成这一转换过程。或者，所有其他CPU通过发送“回写”消息将此缓存行从它们的缓存中剔除（可能是为了给其他缓存行腾出空间），从而使该CPU成为最后一个保留此缓存行的CPU。

Transition (i): 另一颗CPU对仅存于本CPU缓存中的某缓存行内的数据项执行了原子读改写操作，因此本CPU将其从缓存中失效。这一转换由接收“读无效”消息触发，随后本CPU响应为

both 读响应 和 无效确认 消息。

Transition (j): 此CPU将数据项存储到缓存中未包含的缓存行，并因此传输一个“读无效”消息。该CPU无法完成转换，直到收到“读响应”和一整套“无效确认”消息。假设在实际写入完成后，该缓存行会通过转换(b)立即转变为“已修改”状态。

Transition (k): 该CPU加载了一个不在其缓存中的缓存行数据项。CPU发送一个“读取”消息，并在收到相应的“读取响应”后完成过渡。

Transition (l): 另一些CPU对该缓存行中的数据项进行了写操作，但由于该缓存行被其他CPU的缓存（包括当前CPU的缓存）以只读状态持有，因此它保持了只读状态。这一状态转换是由接收“无效”消息触发的，并且此CPU会回应一个“无效确认”消息。

Quick Quiz 4: 硬件是如何处理上述延迟过渡的？

2.4 MESI Protocol Example

现在我们从缓存行的角度来看这个问题，初始时该缓存行的数据位于地址为0的内存中，在一个四CPU系统的各个单线直接映射缓存之间流转。表1展示了这一数据流情况，第一列显示操作序列，第二列显示执行操作的CPU，第三列显示正在进行的操作，接下来的四列表示每个CPU缓存行的状态（包括内存地址和MESI状态），最后两列表示相应内存内容是否是最新的（“V”表示最新，“I”表示不最新）。

初始时，数据将要驻留的CPU缓存行处于“无效”状态，而内存中的数据是有效的。当CPU 0加载该地址的数据时

address 0, it enters the “shared” state in CPU 0’s cache, and is still valid in memory. CPU 3 also loads the data at address 0, so that it is in the “shared” state in both CPUs’ caches, and is still valid in memory. Next CPU 3 loads some other cache line (at address 8), which forces the data at address 0 out of its cache via a writeback, replacing it with the data at address 8. CPU 2 now does a load from address 0, but this CPU realizes that it will soon need to store to it, and so it uses a “read invalidate” message in order to gain an exclusive copy, invalidating it from CPU 3’s cache (though the copy in memory remains up to date). Next CPU 2 does its anticipated store, changing the state to “modified”. The copy of the data in memory is now out of date. CPU 1 does an atomic increment, using a “read invalidate” to snoop the data from CPU 2’s cache and invalidate it, so that the copy in CPU 1’s cache is in the “modified” state (and the copy in memory remains out of date). Finally, CPU 1 reads the cache line at address 8, which uses a “writeback” message to push address 0’s data back out to memory.

Note that we end with data in some of the CPU’s caches.

Quick Quiz 5: What sequence of operations would put the CPUs’ caches all back into the “invalid” state? ☐

3 Stores Result in Unnecessary Stalls

Although the cache structure shown in Figure 1 provides good performance for repeated reads and writes from a given CPU to a given item of data, its performance for the first write to a given cache line is quite poor. To see this, consider Figure 4, which shows a timeline of a write by CPU 0 to a cacheline held in CPU 1’s cache. Since CPU 0 must wait for the cache line to arrive before it can write to it, CPU 0 must stall for an extended period of time.³

³The time required to transfer a cache line from one CPU’s cache to another’s is typically a few orders of magnitude more than that required to execute a simple register-to-register instruction.

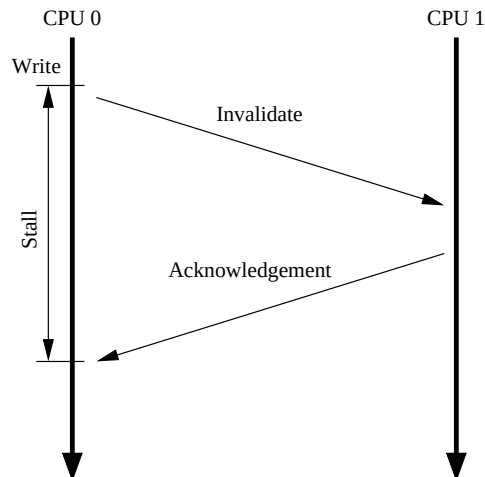


Figure 4: Writes See Unnecessary Stalls

But there is no real reason to force CPU 0 to stall for so long — after all, regardless of what data happens to be in the cache line that CPU 1 sends it, CPU 0 is going to unconditionally overwrite it.

3.1 Store Buffers

One way to prevent this unnecessary stalling of writes is to add “store buffers” between each CPU and its cache, as shown in Figure 5. With the addition of these store buffers, CPU 0 can simply record its write in its store buffer and continue executing. When the cache line does finally make its way from CPU 1 to CPU 0, the data will be moved from the store buffer to the cache line.

However, there are complications that must be addressed, which are covered in the next two sections.

3.2 Store Forwarding

To see the first complication, a violation of self-consistency, consider the following code with variables “a” and “b” both initially zero, and with the cache line containing variable “a” initially owned by CPU 1 and that containing “b” initially owned by CPU 0:

地址为0的数据进入CPU 0缓存的“共享”状态，并且在内存中仍然有效。接着，CPU 3也加载了地址为0的数据，因此该数据同时处于两个CPU缓存的“共享”状态，在内存中依然有效。随后，CPU 3加载了另一个缓存行（地址为8），这导致其通过写回操作将地址为0的数据从缓存中移除，并用地址为8的数据替换它。此时，CPU 2尝试从地址0读取数据，但意识到很快需要对该地址进行存储操作，因此使用“读无效”消息来获取独占副本并使该数据在CPU 3的缓存中失效（尽管内存中的副本仍然有效）。接下来，CPU 2执行了预期的存储操作，将状态更改为“修改”。此时，内存中的数据副本已经过时。接着，CPU 1进行原子增量操作，使用“读无效”消息从CPU 2的缓存中窥探数据并使其失效，使得CPU 1缓存中的数据副本处于“修改”状态（而内存中的副本仍然过时）。最后，CPU 1读取地址为8的缓存行，并通过“写回”消息将地址0的数据推回到内存。

注意，我们在某些CPU缓存中结束了数据的处理。

Quick Quiz 5: 什么样的操作序列可以将所有CPU缓存都重新置为“无效”状态？

3 Stores Result in Unnecessary Stalls

尽管图1所示的缓存结构对于给定CPU对特定数据项进行重复读写时性能良好，但对于首次向给定缓存行写入的情况，其性能却非常差。为了说明这一点，请参见图4，该图展示了CPU 0向位于CPU 1缓存中的某缓存行进行写操作的时间线。由于CPU 0必须等待缓存行到达后才能对其进行写入，因此CPU 0需要长时间停滞。

³The time required to transfer a cache line from one CPU's cache to another's is typically a few orders of magnitude more than that required to execute a simple register-to-register instruction.

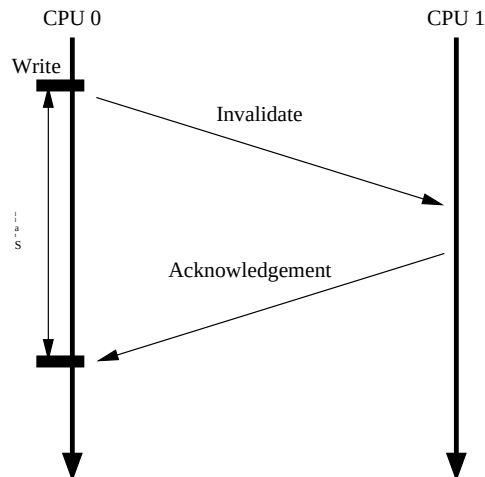


图4：写操作不必要的停顿

但并没有真正的原因非要让CPU 0长时间停顿——毕竟，无论CPU 1发送的缓存行中包含什么数据，CPU 0都会无条件地将其覆盖。

3.1 Store Buffers

一种防止写操作不必要的停滞的方法是在每个CPU与其缓存之间添加“存储缓冲区”，如图5所示。通过增加这些存储缓冲区，CPU 0可以简单地将其写入记录在存储缓冲区内并继续执行。当缓存行最终从CPU 1传输到CPU 0时，数据将从存储缓冲区移动到缓存行。

然而，接下来的两个部分将讨论必须解决的一些复杂问题。

3.2 Store Forwarding

为了看到第一个复杂情况，即自一致性被违反的情况，请考虑以下代码：变量“a”和“b”初始值均为零，并且包含变量“a”的缓存行最初由CPU 1拥有，而包含变量“b”的缓存行最初由CPU 0拥有。

Sequence #	CPU #	Operation	CPU Cache				Memory	
			0	1	2	3	0	8
0		Initial State	-/I	-/I	-/I	-/I	V	V
1	0	Load	0/S	-/I	-/I	-/I	V	V
2	3	Load	0/S	-/I	-/I	0/S	V	V
3	0	Writeback	8/S	-/I	-/I	0/S	V	V
4	2	RMW	8/S	-/I	0/E	-/I	V	V
5	2	Store	8/S	-/I	0/M	-/I	I	V
6	1	Atomic Inc	8/S	0/M	-/I	-/I	I	V
7	1	Writeback	8/S	8/S	-/I	-/I	V	V

Table 1: Cache Coherence Example

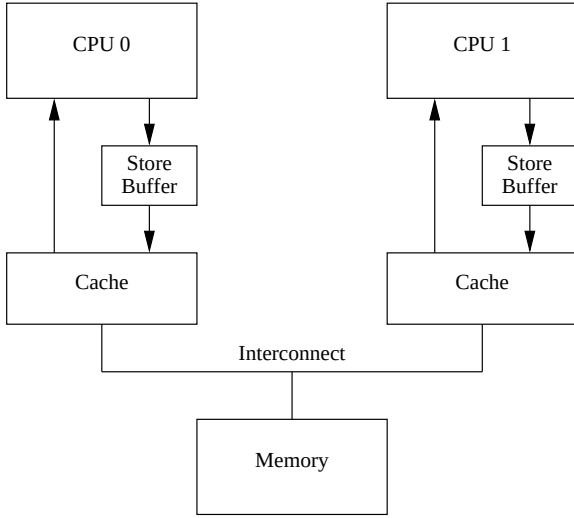


Figure 5: Caches With Store Buffers

```

1  a = 1;
2  b = a + 1;
3  assert(b == 2);

```

One would not expect the assertion to fail. However, if one were foolish enough to use the very simple architecture shown in Figure 5, one would be surprised. Such a system could potentially see the following sequence of events:

1. CPU 0 starts executing the `a=1`.
2. CPU 0 looks “a” up in the cache, and finds that

it is missing.

3. CPU 0 therefore sends a “read invalidate” message in order to get exclusive ownership of the cache line containing “a”.
4. CPU 0 records the store to “a” in its store buffer.
5. CPU 1 receives the “read invalidate” message, and responds by transmitting the cache line and removing that cacheline from its cache.
6. CPU 0 starts executing the `b=a+1`.
7. CPU 0 receives the cache line from CPU 1, which still has a value of zero for “a”.
8. CPU 0 loads “a” from its cache, finding the value zero.
9. CPU 0 applies the entry from its store queue to the newly arrived cache line, setting the value of “a” in its cache to one.
10. CPU 0 adds one to the value zero loaded for “a” above, and stores it into the cache line containing “b” (which we will assume is already owned by CPU 0).
11. CPU 0 executes `assert(b==2)`, which fails.

The problem is that we have two copies of “a”, one in the cache and the other in the store buffer.

This example breaks a very important guarantee, namely that each CPU will always see its own operations as if they happened in program order.

Sequence #	CPU #	Operation	CPU Cache				Memory	
			0	1	2	3	0	8
0		Initial State	-/I	-/I	-/I	-/I	V	V
1	0	Load	0/S	-/I	-/I	-/I	V	V
2	3	Load	0/S	-/I	-/I	0/S	V	V
3	0	Writeback	8/S	-/I	-/I	0/S	V	V
4	2	RMW	8/S	-/I	0/E	-/I	V	V
5	2	Store	8/S	-/I	0/M	-/I	1	V
6	1	Atomic Inc	8/S	0/M	-/I	-/I	1	V
7	1	Writeback	8/S	8/S	-/I	-/I	V	V

表1：缓存一致性示例

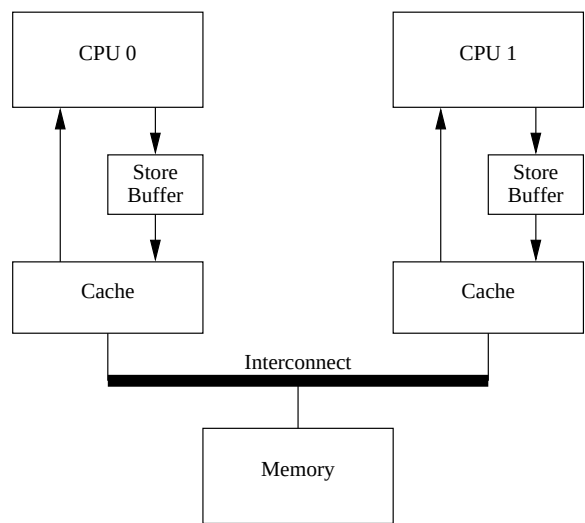


图5：带有存储缓冲区的缓存

```

1  a = 1;
2  b = a + 1;
3  assert(b == 2);

```

人们不会预期该断言会失败。然而，如果有人愚蠢到使用图5所示的非常简单的架构，他们将会感到惊讶。这样的系统可能会遇到以下事件序列：

1. CPU 0 开始执行 `a=1`。
2. CPU 0 在缓存中查找 “a”， 并发现

3. 因此，CPU 0 发送一个“读取无效”的消息，以获得包含“a”的缓存行的独占所有权。 4. CPU 0 将对“a”的存储记录在其存储缓冲区中。 5. CPU 1 收到“读取无效”消息后，通过传输该缓存行并从中移除该缓存行来响应。 6. CPU 0 开始执行 `b=a+1`。 7. CPU 0 从 CPU 1 处接收缓存行，此时“a”的值仍为零。 8. CPU 0 从其缓存中加载“a”，发现其值为零。 9. CPU 0 将存储队列中的条目应用到新到达的缓存行上，将其缓存中“a”的值设置为一。

请提供需要翻译的英文学术文本内容。目前提供的信息中缺少具体的存储到包含“b”的缓存行中（假设该缓存行已被CPU 0 所拥有）。

请提供需要翻译的英文学术文本内容。目前提供的信息中缺少具体的

我们有两个“a”的副本，一个在缓存中，另一个在存储缓冲区中。

这个例子打破了非常重要的一条保证，即每个CPU总是会看到自己的操作仿佛是按程序顺序发生的。

This guarantee is violently counter-intuitive to software types, so much so that the hardware guys took pity and implemented “store forwarding”, where each CPU refers to (or “snoops”) its store buffer as well as its cache when performing loads, as shown in Figure 6. In other words, a given CPU’s stores are directly forwarded to its subsequent loads, without having to pass through the cache.

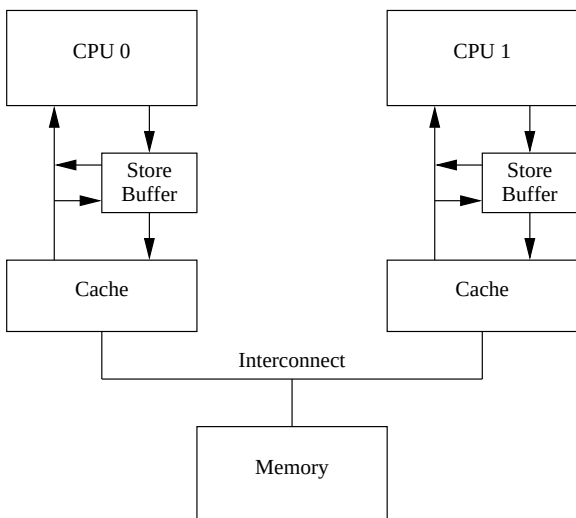


Figure 6: Caches With Store Forwarding

With store forwarding in place, item 8 in the above sequence would have found the correct value of 1 for “a” in the store buffer, so that the final value of “b” would have been 2, as one would hope.

3.3 Store Buffers and Memory Barriers

To see the second complication, a violation of global memory ordering, consider the following code sequences with variables “a” and “b” initially zero:

```

1 void foo(void)
2 {
3   a = 1;
4   b = 1;
5 }
6
7 void bar(void)
8 {
9   while (b == 0) continue;
10  assert(a == 1);
11 }
  
```

Suppose CPU 0 executes `foo()` and CPU 1 executes `bar()`. Suppose further that the cache line containing “a” resides only in CPU 1’s cache, and that the cache line containing “b” is owned by CPU 0. Then the sequence of operations might be as follows:

1. CPU 0 executes `a=1`. The cache line is not in CPU 0’s cache, so CPU 0 places the new value of “a” in its store buffer and transmits a “read invalidate” message.
2. CPU 1 executes `while(b==0)continue`, but the cache line containing “b” is not in its cache. It therefore transmits a “read” message.
3. CPU 0 executes `b=1`. It already owns this cache line (in other words, the cache line is already in either the “modified” or the “exclusive” state), so it stores the new value of “b” in its cache line.
4. CPU 0 receives the “read” message, and transmits the cache line containing the now-updated value of “b” to CPU 1, also marking the line as “shared” in its own cache.
5. CPU 1 receives the cache line containing “b” and installs it in its cache.
6. CPU 1 can now finish executing `while(b==0)continue`, and since it finds that the value of “b” is 1, it proceeds to the next statement.
7. CPU 1 executes the `assert(a==1)`, and, since CPU 1 is working with the old value of “a”, this assertion fails.
8. CPU 1 receives the “read invalidate” message, and transmits the cache line containing “a” to

这种保证对于软件类型来说是极其违背直觉的，以至于硬件工程师出于同情实现了“存储转发”，即每个CPU在执行加载操作时会参考（或称为“监听”）其存储缓冲区以及缓存，如图6所示。换句话说，某个CPU的存储操作可以直接转发到后续的加载操作中，而无需通过缓存传递。

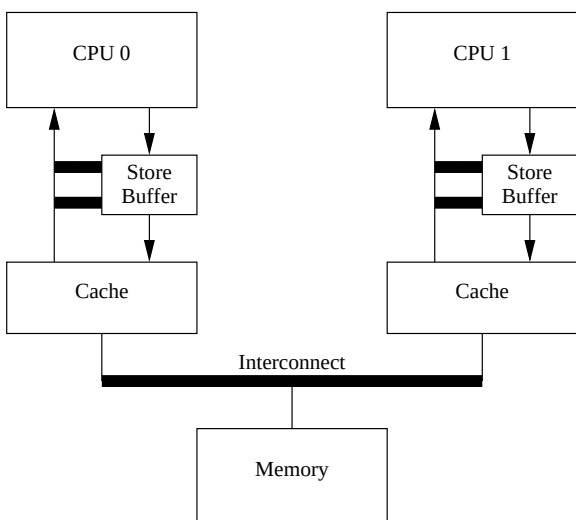


图6：带有存储转发的缓存

启用store转发后，上述序列中的第8项会在存储缓冲区中找到变量“a”的正确值1，因此最终变量“b”的值将会是2，如预期的那样。

```

1 void foo(void)
2 {
3     a = 1;
4     b = 1;
5 }
6
7 void bar(void)
8 {
9     while (b == 0) continue;
10    assert(a == 1);
11 }

```

假设CPU 0执行foo()，而CPU 1执行bar()。进一步假设包含“a”的缓存行仅存在于CPU 1的缓存中，包含“b”的缓存行由CPU 0拥有。那么操作序列可能是以下这样的：

1. CPU 0 执行 `a=1`。由于缓存行不在CPU 0的缓存中，因此CPU 0将“a”的新值放入其存储缓冲区，并发送一个“读取失效”消息。
2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。
3. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），所以它将“b”的新值存储在其缓存行中。
4. CPU 0 收到“读取”消息，并将其包含已更新的“b”值的缓存行发送给CPU 1，在其自己的缓存中标记该行作为“共享”。
5. CPU 1 接收包含“b”的缓存行并将其安装在其缓存中。
6. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。
7. CPU 1 执行 `assert(a==1)`，但由于其使用的是旧的“a”值，此断言失败。
8. CPU 1 收到“读取失效”消息，并将其包含“a”的缓存行发送给

3.3 Store Buffers and Memory Barriers

为了看到第二个复杂情况，即全局内存顺序的违反，请考虑以下代码序列，其中变量“a”和“b”初始值为零：

CPU 0 and invalidates this cache line from its own cache. But it is too late.

9. CPU 0 receives the cache line containing “a” and applies the buffered store just in time to fall victim to CPU 1’s failed assertion.

The hardware designers cannot help directly here, since the CPUs have no idea which variables are related, let alone how they might be related. Therefore, the hardware designers provide memory-barrier instructions to allow the software to tell the CPU about such relations. The program fragment must be updated to contain the memory barrier:

```
1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }
```

The memory barrier `smp_mb()` will cause the CPU to flush its store buffer before applying subsequent stores to their cache lines. The CPU could either simply stall until the store buffer was empty before proceeding, or it could use the store buffer to hold subsequent stores until all of the prior entries in the store buffer had been applied.

With this latter approach the sequence of operations might be as follows:

1. CPU 0 executes `a=1`. The cache line is not in CPU 0’s cache, so CPU 0 places the new value of “a” in its store buffer and transmits a “read invalidate” message.
2. CPU 1 executes `while(b==0)continue`, but the cache line containing “b” is not in its cache. It therefore transmits a “read” message.
3. CPU 0 executes `smp_mb()`, and marks all current store-buffer entries (namely, the `a=1`).
4. CPU 0 executes `b=1`. It already owns this cache line (in other words, the cache line is already in either the “modified” or the “exclusive” state), but there is a marked entry in the store buffer. Therefore, rather than store the new value of “b” in the cache line, it instead places it in the store buffer (but in an *unmarked* entry).
5. CPU 0 receives the “read” message, and transmits the cache line containing the original value of “b” to CPU 1. It also marks its own copy of this cache line as “shared”.
6. CPU 1 receives the cache line containing “b” and installs it in its cache.
7. CPU 1 can now finish executing `while(b==0)continue`, but since it finds that the value of “b” is still 0, it repeats the `while` statement. The new value of “b” is safely hidden in CPU 0’s store buffer.
8. CPU 1 receives the “read invalidate” message, and transmits the cache line containing “a” to CPU 0 and invalidates this cache line from its own cache.
9. CPU 0 receives the cache line containing “a” and applies the buffered store.
10. Since the store to “a” was the only entry in the store buffer that was marked by the `smp_mb()`, CPU 0 can also store the new value of “b” — except for the fact that the cache line containing “b” is now in “shared” state.
11. CPU 0 therefore sends an “invalidate” message to CPU 1.
12. CPU 1 receives the “invalidate” message, invalidates the cache line containing “b” from its cache, and sends an “acknowledgement” message to CPU 0.
13. CPU 1 executes `while(b==0)continue`, but the cache line containing “b” is not in its cache. It therefore transmits a “read” message to CPU 0.

CPU 0 将其缓存中的这条缓存行置为无效。但为时已晚。

9. CPU 0 接收到了包含“a”的缓存行，并及时应用了缓冲存储操作，恰好成为CPU 1失败断言的受害者。

硬件设计师在此无能为力，因为CPU无法确定哪些变量是相关的，更不用说它们是如何关联的。因此，硬件设计师提供了内存屏障指令，允许软件向CPU告知这些关系。程序片段必须更新以包含内存屏障：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }
```

内存屏障 `smp_mb()` 将导致 CPU 在应用后续存储操作之前清空其写缓冲区。CPU 可以选择简单地暂停执行，直到写缓冲区为空后再继续，或者使用写缓冲区来暂存后续的存储操作，直至先前的所有条目都被应用到缓存行中。

采用后一种方法，操作序列可能是以下这样的：

1. CPU 0 执行 `a=1`。由于缓存行不在CPU 0的缓存中，因此CPU 0将“a”的新值放入其存储缓冲区，并发送一条“读取失效”消息。2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一条“读取”消息。3. CPU 0 执行 `smp_mb()`，并标记所有当前的存储缓冲区条目（即，`a=1`）。

4. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），但存储缓冲区中有一个标记条目。因此，CPU 0 不将新值“b”存储在缓存行中，而是将其放入存储缓冲区（但在一个 *unmarked* 条目中）。

CPU 0 收到“读取”消息，并将包含变量“b”原始值的缓存行传输给 CPU 1。它还将自己持有的该缓存行标记为“共享”。

6. CPU 1 接收包含“b”的缓存行并将其安装到自己的缓存中。

7. CPU 1 现在可以完成执行 `while(b==0)continue`，但由于发现“b”的值仍然是 0，它重复执行了 `while` 语句。新的“b”值安全地隐藏在 CPU 0 的存储缓冲区中。8. CPU 1 收到“读取失效”消息，并将包含“a”的缓存行传输给 CPU 0 并从自己的缓存中使该缓存行失效。9. CPU 0 接收包含“a”的缓存行并应用了存储缓冲区中的写操作。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的缓冲区中唯一的条目，因此CPU 0也可以存储“b”的新值——除非包含“b”的缓存行现在处于“共享”状态。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的“b”的缓存行置为无效，并向CPU 0 发送“确认”消息。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的“”的缓存行不在其缓存中。因此，它向CPU 0 发送一个“读取”消息。

14. CPU 0 receives the “acknowledgement” message, and puts the cache line containing “b” into the “exclusive” state. CPU 0 now stores the new value of “b” into the cache line.
15. CPU 0 receives the “read” message, and transmits the cache line containing the original value of “b” to CPU 1. It also marks its own copy of this cache line as “shared”.
16. CPU 1 receives the cache line containing “b” and installs it in its cache.
17. CPU 1 can now finish executing `while(b==0) continue`, and since it finds that the value of “b” is 1, it proceeds to the next statement.
18. CPU 1 executes the `assert(a==1)`, but the cache line containing “a” is no longer in its cache. Once it gets this cache from CPU 0, it will be working with the up-to-date value of “a”, and the assertion therefore passes.

As you can see, this process involves no small amount of bookkeeping. Even something intuitively simple, like “load the value of a” can involve lots of complex steps in silicon.

4 Store Sequences Result in Unnecessary Stalls

Unfortunately, each store buffer must be relatively small, which means that a CPU executing a modest sequence of stores can fill its store buffer (for example, if all of them result in cache misses). At that point, the CPU must once again wait for invalidations to complete in order to drain its store buffer before it can continue executing. This same situation can arise immediately after a memory barrier, when *all* subsequent store instructions must wait for invalidations to complete, regardless of whether or not these stores result in cache misses.

This situation can be improved by making invalidate acknowledge messages arrive more quickly. One way of accomplishing this is to use per-CPU queues of invalidate messages, or “invalidate queues”.

4.1 Invalidate Queues

One reason that invalidate acknowledge messages can take so long is that they must ensure that the corresponding cache line is actually invalidated, and this invalidation can be delayed if the cache is busy, for example, if the CPU is intensively loading and storing data, all of which resides in the cache. In addition, if a large number of invalidate messages arrive in a short time period, a given CPU might fall behind in processing them, thus possibly stalling all the other CPUs.

However, the CPU need not actually invalidate the cache line before sending the acknowledgement. It could instead queue the invalidate message with the understanding that the message will be processed before the CPU sends any further messages regarding that cache line.

4.2 Invalidate Queues and Invalidate Acknowledge

Figure 7 shows a system with invalidate queues. A CPU with an invalidate queue may acknowledge an invalidate message as soon as it is placed in the queue, instead of having to wait until the corresponding line is actually invalidated. Of course, the CPU must refer to its invalidate queue when preparing to transmit invalidation messages — if an entry for the corresponding cache line is in the invalidate queue, the CPU cannot immediately transmit the invalidate message; it must instead wait until the invalidate-queue entry has been processed.

Placing an entry into the invalidate queue is essentially a promise by the CPU to process that entry before transmitting any MESI protocol messages regarding that cache line. As long as the corresponding data structures are not highly contended, the CPU will rarely be inconvenienced by such a promise.

However, the fact that invalidate messages can be buffered in the invalidate queue provides additional opportunity for memory-misordering, as discussed in the next section.

14. CPU 0 收到“确认”消息，并将包含“b”的缓存行置为“独占”状态。CPU 0 现在将“b”的新值存储到该缓存行中。

15. CPU 0 收到“读取”消息，并将包含变量“b”原始值的缓存行传输给CPU 1。它还将自己持有的该缓存行标记为“共享”。

16. CPU 1 接收包含“b”的缓存行并将其安装到自己的缓存中。

CPU 1 现在可以完成执行 `while(b==0) continue`，并且由于它发现变量“b”的值为 1，因此继续执行下一条语句。

18. CPU 1 执行了 `assert(a==1)`，但包含“a”的缓存行已不在其缓存中。一旦从CPU 0 获取该缓存行后，它将使用最新的“a”值，并且断言因此通过验证。

如您所见，这个过程涉及大量的记录工作。即使是直观上看似简单的操作，比如“加载a的值”，在硅片实现中也可能包含许多复杂的步骤。

4 Store Sequences Result in Unnecessary Stalls

不幸的是，每个存储缓冲区必须相对较小，这意味着执行一串适度长度的存储指令时，CPU可能会填满其存储缓冲区（例如，如果所有这些指令都导致缓存未命中）。此时，CPU必须再次等待无效操作完成以清空其存储缓冲区，才能继续执行。这种情况也可能在内存屏障之后立即出现，当`all`后续的存储指令必须等待无效操作完成，无论这些存储指令是否导致缓存未命中。

这种情况可以通过使无效化确认消息更快到达来改善。一种实现方法是使用每个CPU的无效化消息队列，或称为“无效化队列”。

4.1 Invalidate Queues

无效确认消息耗时较长的一个原因是，它们必须确保对应的缓存行确实被清除了。如果缓存忙于处理数据（例如，CPU正在进行大量数据的加载和存储操作），这些清除操作可能会被延迟。此外，在短时间内收到大量无效消息的情况下，某个CPU可能会落后于处理这些消息，从而可能导致所有其他CPU都陷入停滞状态。

然而，CPU并不需要在发送确认之前实际使缓存行失效。它也可以将失效消息排队，理解为该消息将在CPU发送有关该缓存行的任何进一步消息之前被处理。

4.2 Invalidate Queues and Invalidate Acknowledge

图7展示了一个带有无效队列的系统。一个配备了无效队列的CPU可以在收到无效消息时立即将其放入队列中进行确认，而无需等待对应的缓存行实际被无效化。当然，在准备发送无效消息之前，CPU必须检查其无效队列——如果对应缓存行的条目存在于无效队列中，则CPU不能立即传输该无效消息；它必须等到无效队列中的该项被处理完毕后才能继续。

将条目放入无效队列本质上是CPU的一种承诺，即在传输任何与该缓存行相关的MESI协议消息之前处理该条目。只要相应的数据结构不会高度竞争，CPU很少会因这种承诺而受到困扰。

然而，无效消息可以在无效队列中缓存这一事实为内存乱序访问提供了额外的机会，如下一节所述。

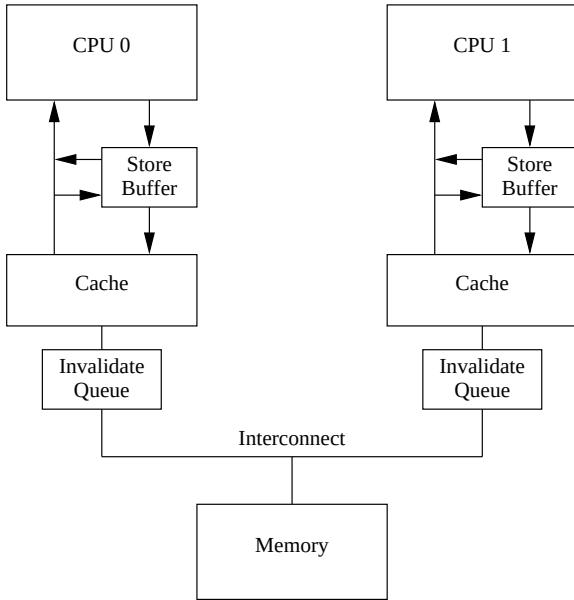


Figure 7: Caches With Invalidate Queues

4.3 Invalidate Queues and Memory Barriers

Suppose the values of “a” and “b” are initially zero, that “a” is replicated read-only (MESI “shared” state), and that “b” is owned by CPU 0 (MESI “exclusive” or “modified” state). Then suppose that CPU 0 executes `foo()` while CPU 1 executes function `bar()` in the following code fragment:

```

1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }
```

Then the sequence of operations might be as fol-

lows:

1. CPU 0 executes `a=1`. The corresponding cache line is read-only in CPU 0’s cache, so CPU 0 places the new value of “a” in its store buffer and transmits an “invalidate” message in order to flush the corresponding cache line from CPU 1’s cache.
2. CPU 1 executes `while(b==0)continue`, but the cache line containing “b” is not in its cache. It therefore transmits a “read” message.
3. CPU 0 executes `b=1`. It already owns this cache line (in other words, the cache line is already in either the “modified” or the “exclusive” state), so it stores the new value of “b” in its cache line.
4. CPU 0 receives the “read” message, and transmits the cache line containing the now-updated value of “b” to CPU 1, also marking the line as “shared” in its own cache.
5. CPU 1 receives the “invalidate” message for “a”, places it into its invalidate queue, and transmits an “invalidate acknowledge” message to CPU 0. Note that the old value of “a” still remains in CPU 1’s cache.
6. CPU 1 receives the cache line containing “b” and installs it in its cache.
7. CPU 1 can now finish executing `while(b==0)continue`, and since it finds that the value of “b” is 1, it proceeds to the next statement.
8. CPU 1 executes the `assert(a==1)`, and, since the old value of “a” is still in CPU 1’s cache, this assertion fails.
9. CPU 1 processes the queued “invalidate” message, and invalidates the cache line containing “a” from its own cache. But it is too late.
10. CPU 0 receives the “invalidate acknowledge” message for “a” from CPU 1, and therefore applies the buffered store just in time to fall victim to CPU 1’s failed assertion.

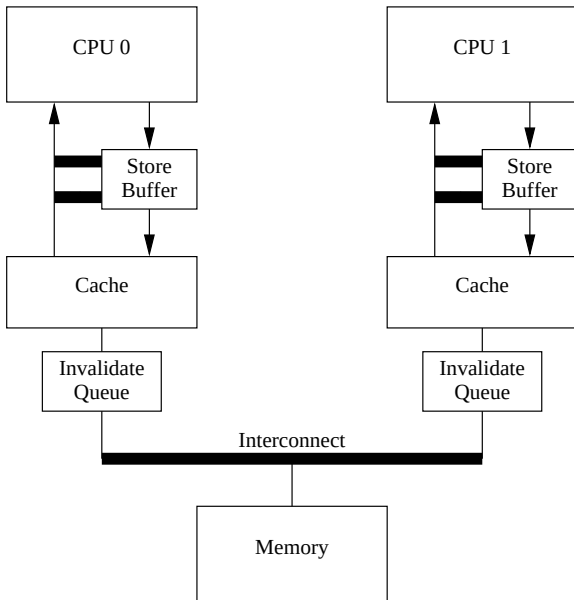


图7: 带有无效队列的缓存

4.3 Invalidate Queues and Memory Barriers

假设变量“a”和“b”的初始值均为零，“a”被只读复制（MESI共享状态），而“b”由CPU 0独占（MESI专有或修改状态）。然后假定CPU 0执行代码片段中的foo()，而CPU 1执行函数bar()：

```

1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }

```

然后操作序列可能是如下所示：

低频：

CPU 0 执行 `a=1`。在 CPU 0 的缓存中，对应的缓存行是只读的，因此 CPU 0 将“a”的新值放入其存储缓冲区，并发送一个“无效”消息以从 CPU 1 的缓存中清除相应的缓存行。

2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。

3. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），因此它将其缓存行中的“b”的新值存储下来。

4. CPU 0 收到“读取”消息，并将包含已更新值“b”的缓存行传输给CPU 1，同时在自己的缓存中标记该行状态为“共享”。5. CPU 1 收到针对“a”的“失效”消息，将其放入失效队列中，并向CPU 0 发送一个“失效确认”消息。需要注意的是，“a”的旧值仍然保留在CPU 1的缓存中。6. CPU 1 接收包含“b”的缓存行并安装到自己的缓存中。7. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。8. CPU 1 执行 `assert(a==1)`，但由于“a”的旧值仍然保留在CPU 1的缓存中，此断言失败。9. CPU 1 处理队列中的“失效”消息，并从自己的缓存中清除包含“a”的缓存行。但此时已经太迟了。

CPU 0 收到了来自自身的“无效确认”消息（针对“A”），因此刚刚缓冲的存储操作正好赶在了CPU 1 的失败断言之下，成了受害者。

Once again, the CPU designers cannot do much about this situation, since the hardware does not know what relationships there might be among what to the CPU are just different piles of bits. However, the memory-barrier instructions can interact with the invalidate queue, so that when a given CPU executes a memory barrier, it marks all the entries currently in its invalidate queue, and forces any subsequent load to wait until all marked entries have been applied to the CPU's cache. Therefore, we can add a memory barrier as follows:

```

1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_mb();
12    assert(a == 1);
13 }
```

With this change, the sequence of operations might be as follows:

1. CPU 0 executes `a=1`. The corresponding cache line is read-only in CPU 0's cache, so CPU 0 places the new value of "a" in its store buffer and transmits an "invalidate" message in order to flush the corresponding cache line from CPU 1's cache.
2. CPU 1 executes `while(b==0)continue`, but the cache line containing "b" is not in its cache. It therefore transmits a "read" message.
3. CPU 0 executes `b=1`. It already owns this cache line (in other words, the cache line is already in either the "modified" or the "exclusive" state), so it stores the new value of "b" in its cache line.
4. CPU 0 receives the "read" message, and transmits the cache line containing the now-updated value of "b" to CPU 1, also marking the line as "shared" in its own cache.
5. CPU 1 receives the "invalidate" message for "a", places it into its invalidate queue, and transmits an "invalidate acknowledge" message to CPU 0. Note that the old value of "a" still remains in CPU 1's cache.
6. CPU 1 receives the cache line containing "b" and installs it in its cache.
7. CPU 1 can now finish executing `while(b==0)continue`, and since it finds that the value of "b" is 1, it proceeds to the next statement.
8. CPU 1 executes the `smp_mb()`, marking the entry in its invalidate queue.
9. CPU 1 executes the `assert(a==1)`, but since there is a marked entry for the cache line containing "a" in the invalidate queue, CPU 1 must stall this load until that entry in the invalidate queue has been applied.
10. CPU 1 processes the "invalidate" message, removing the cacheline containing "a" from its cache.
11. CPU 1 is now free to load the value of "a", but since this results in a cache miss, it must send a "read" message to fetch the corresponding cache line.
12. CPU 0 receives the "invalidate acknowledge" message for "a" from CPU 1, and therefore applies the buffered store, changing the MESI state of the corresponding cache line to "modified".
13. CPU 0 receives the "read" message for "a" from CPU 1, and therefore changes the state of the corresponding cache line to "shared", and transmits the cache line to CPU 1.
14. CPU 1 receives the cache line containing "a", and can therefore do the load. Since this load returns the updated value of "a", the assertion passes.

With much passing of MESI messages, the CPUs arrive at the correct answer.

再次，CPU设计者对此无能为力，因为硬件无法知晓那些对CPU而言只是不同位堆栈之间的关系。然而，内存屏障指令可以与失效队列交互，因此当某个CPU执行内存屏障时，它会标记当前存在于其失效队列中的所有条目，并强制后续的任何加载操作等待，直到所有已标记的条目都被应用到该CPU的缓存中。因此，我们可以添加如下所示的内存屏障：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_mb();
12    assert(a == 1);
13 }
```

随着这一改动，操作序列可能如下所示：

1. CPU 0 执行 `a=1`。在 CPU 0 的缓存中，包含“a”的缓存行是只读的，因此 CPU 0 将“a”的新值放入其存储缓冲区，并发送一个“无效”消息以清除 CPU 1 缓存中的相应缓存行。2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。3. CPU 0 执行 `b=1`。它已经拥有此缓存行（也就是说，该缓存行已经在“修改”或“独占”状态），所以它将“b”的新值存储在其缓存行中。4. CPU 0 收到“读取”消息，并将其包含已更新的“b”值的缓存行发送给 CPU 1，在其自身的缓存中标记该行作为“共享”。

5. CPU 1 收到针对“a”的“无效”消息，将其放入无效队列，并向 CPU 0 发送一个“无效确认”消息。需要注意的是，“a”的旧值仍然保留在 CPU 1 的缓存中。6. CPU 1 接收到包含“b”的缓存行并将其安装到自己的缓存中。7. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。8. CPU 1 执行 `smp_mb()`，在其无效队列中标记该条目。9. 当前，CPU 1 执行 `assert(a==1)`，但由于其无效队列中包含针对“a”缓存行的标记条目，因此必须暂停此加载操作，直到该条目被应用为止。10. CPU 1 处理“无效”消息，从其缓存中移除包含“a”的缓存行。11. 现在，CPU 1 可以加载“a”的值了。但由于这导致缓存未命中，它必须发送一个“读取”消息来获取相应的缓存行。12. CPU 0 收到来自 CPU 1 的针对“a”的“无效确认”消息，并因此应用缓冲存储操作，将相应缓存行的MESI状态更改为“修改”。13. CPU 0 接收到来自 CPU 1 针对“a”的“读取”请求，并因此更改了相应缓存行的状态为“共享”，并将该缓存行传输给 CPU 1。14. CPU 1 收到包含“a”的缓存行，现在可以执行加载操作。由于此加载返回更新后的“a”的值，断言通过验证。

经过大量MESI消息的传递，CPU最终得出了正确的结果。

5 Read and Write Memory Barriers

In the previous section, memory barriers were used to mark entries in both the store buffer and the invalidate queue. But in our code fragment, `foo()` had no reason to do anything with the invalidate queue, and `bar()` similarly had no reason to do anything with the store queue.

Many CPU architectures therefore provide weaker memory-barrier instructions that do only one or the other of these two. Roughly speaking, a “read memory barrier” marks only the invalidate queue and a “write memory barrier” marks only the store buffer, while a full-fledged memory barrier does both.

The effect of this is that a read memory barrier orders only loads on the CPU that executes it, so that all loads preceding the read memory barrier will appear to have completed before any load following the read memory barrier. Similarly, a write memory barrier orders only stores, again on the CPU that executes it, and again so that all stores preceding the write memory barrier will appear to have completed before any store following the write memory barrier. A full-fledged memory barrier orders both loads and stores, but again only on the CPU executing the memory barrier.

If we update `foo` and `bar` to use read and write memory barriers, they appear as follows:

```
1 void foo(void)
2 {
3     a = 1;
4     smp_wmb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_rmb();
12    assert(a == 1);
13 }
```

Some computers have even more flavors of memory barriers, but understanding these three variants will provide a good introduction to memory barriers in

general.

6 Example Memory-Barrier Sequences

This section presents some seductive but subtly broken uses of memory barriers. Although many of them will work most of the time, and some will work all the time on some specific CPUs, these uses must be avoided if the goal is to produce code that works reliably on all CPUs. To help us better see the subtle breakage, we first need to focus on an ordering-hostile architecture.

6.1 Ordering-Hostile Architecture

Paul has come across a number of ordering-hostile computer systems, but the nature of the hostility has always been extremely subtle, and understanding it has required detailed knowledge of the specific hardware. Rather than picking on a specific hardware vendor, and as a presumably attractive alternative to dragging the reader through detailed technical specifications, let us instead design a mythical but maximally memory-ordering-hostile computer architecture.⁴

This hardware must obey the following ordering constraints [16, 17]:

1. Each CPU will always perceive its own memory accesses as occurring in program order.
2. CPUs will reorder a given operation with a store only if the two operations are referencing different locations.
3. All of a given CPU’s loads preceding a read memory barrier (`smp_rmb()`) will be perceived by all CPUs to precede any loads following that read memory barrier.

⁴Readers preferring a detailed look at real hardware architectures are encouraged to consult CPU vendors’ manuals [19, 1, 8, 6, 15, 20, 10, 9, 13] or Gharachorloo’s dissertation [4].

5 Read and Write Memory Barriers

在上一节中，内存屏障被用来标记存储缓冲区和无效队列中的条目。但在我们的代码片段中，`foo()` 没有必要与无效队列进行任何操作，类似地，`bar()` 也没有必要与存储队列进行任何操作。

许多CPU架构因此提供了较弱的内存屏障指令，这些指令只执行上述两种操作中的某一种。大致来说，“读内存屏障”仅标记无效队列，“写内存屏障”仅标记存储缓冲区，而完整的内存屏障则同时进行这两项操作。

这种效果是，读内存屏障仅对执行它的CPU上的加载操作进行排序，因此所有在读内存屏障之前的加载操作都将看似已经完成，然后再执行任何后续的加载操作。同样地，写内存屏障仅对执行它的CPU上的存储操作进行排序，并且所有在写内存屏障之前的存储操作都将看似已经完成，然后再执行任何后续的存储操作。一个完整的内存屏障则同时对加载和存储操作进行排序，但仍然只针对执行该内存屏障的CPU。

如果我们将 `foo` 和 `bar` 更新为使用读写内存屏障，则它们将变为如下形式：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_wmb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_rmb();
12    assert(a == 1);
13 }
```

一些计算机甚至具有更多种类的内存屏障，但理解这三种变体将为了了解内存屏障提供良好的入门介绍。

通用。

6 Example Memory-Barrier Sequences

本节介绍了一些引人注目但微妙地存在问题的内存屏障用法。尽管其中许多方法在大多数情况下都能正常工作，某些甚至会在特定的CPU上始终如一地运行，但如果目标是编写能够在所有CPU上可靠运行的代码，则必须避免使用这些方法。为了更好地揭示这种细微的问题，我们首先需要关注一个敌对顺序的架构。

6.1 Ordering-Hostile Architecture

保罗遇到过多种敌对有序性的计算机系统，但这种敌对性总是极其微妙，理解它需要详细了解特定的硬件。为了避免针对某个具体的硬件供应商，并且作为吸引读者的一种替代方案，而不是让他们陷入详细的规格说明中，让我们设计一种神话般的、但却具有最大内存顺序敌对性的计算机架构。⁴

该硬件必须遵守以下顺序约束条件 [16, 17]：

每个CPU总是会将其自身的内存访问视为按程序顺序发生的。

2. CPU仅在两个操作引用不同位置时，才会重新排序给定的存储操作。

3. 给定CPU的所有在读内存屏障（`smp_rmb()`）之前的加载操作，都将被所有其他CPU感知为早于该读内存屏障之后的任何加载操作。

⁴Readers preferring a detailed look at real hardware architectures are encouraged to consult CPU vendors' manuals [19, 1, 8, 6, 15, 20, 10, 9, 13] or Gharachorloo's dissertation [4].

CPU 0	CPU 1	CPU 2
<pre>a=1; smp_wmb(); b=1;</pre>	<pre>while(b==0); c=1;</pre>	<pre>z=c; smp_rmb(); x=a; assert(z==0 x==1);</pre>

Table 2: Memory Barrier Example 1

4. All of a given CPU's stores preceding a write memory barrier (`smp_wmb()`) will be perceived by all CPUs to precede any stores following that write memory barrier.
5. All of a given CPU's accesses (loads and stores) preceding a full memory barrier (`smp_mb()`) will be perceived by all CPUs to precede any accesses following that memory barrier.

Quick Quiz 6: Does the guarantee that each CPU sees its own memory accesses in order also guarantee that each user-level thread will see its own memory accesses in order? Why or why not? $\square$

Imagine a large non-uniform cache architecture (NUCA) system that, in order to provide fair allocation of interconnect bandwidth to CPUs in a given node, provided per-CPU queues in each node's interconnect interface, as shown in Figure 8. Although a given CPU's accesses are ordered as specified by memory barriers executed by that CPU, however, the relative order of a given pair of CPUs' accesses could be severely reordered, as we will see.⁵

6.2 Example 1

Figure 2 shows three code fragments, executed concurrently by CPUs 1, 2, and 3. Each of "a", "b", and "c" are initially zero.

⁵Any real hardware architect or designer will no doubt be loudly calling for Ralph on the porcelain intercom, as they just might be just a bit upset about the prospect of working out which queue should handle a message involving a cache line that both CPUs accessed, to say nothing of the many races that this example poses. All I can say is "Give me a better example".

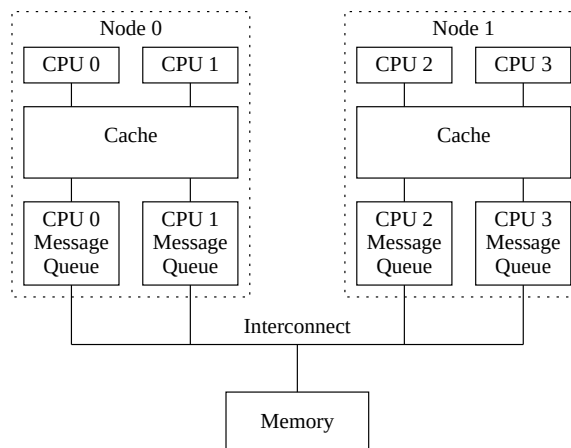


Figure 8: Example Ordering-Hostile Architecture

Suppose CPU 0 recently experienced many cache misses, so that its message queue is full, but that CPU 1 has been running exclusively within the cache, so that its message queue is empty. Then CPU 0's assignment to "a" and "b" will appear in Node 0's cache immediately (and thus be visible to CPU 1), but will be blocked behind CPU 0's prior traffic. In contrast, CPU 1's assignment to "c" will sail through CPU 1's previously empty queue. Therefore, CPU 2 might well see CPU 1's assignment to "c" before it sees CPU 0's assignment to "a", causing the assertion to fire, despite the memory barriers.

In theory, portable code could not on this example coding technique, however, in practice it actually does work on all mainstream computer systems.

Quick Quiz 7: Could this code be fixed by inserting a memory barrier between CPU 1's "while" and assignment to "c"? Why or why not? $\square$

6.3 Example 2

Figure 3 shows three code fragments, executed concurrently by CPUs 1, 2, and 3. Both "a" and "b" are initially zero.

Again, suppose CPU 0 recently experienced many cache misses, so that its message queue is full, but that CPU 1 has been running exclusively within the

CPU 0	CPU 1	CPU 2
<pre>a=1; smp_wmb(); b=1;</pre>	<pre>while(b==0); c=1;</pre>	<pre>z=c; smp_rmb(); x=a; assert(z==0 x==1);</pre>

表2：内存屏障示例1

4. 给定CPU在写内存屏障（`smp_wmb()`）之前的所有存储操作，都将被所有其他CPU感知为发生在该写内存屏障之后的任何存储操作之前。

5. 给定CPU的所有访问操作（加载和存储）在全内存屏障（`smp_mb()`）之前进行，所有其他CPU都会感知到这些操作发生在任何跨越该内存屏障的操作之前。

Quick Quiz 6: 每个CPU都能按顺序看到自己的内存访问，这是否也保证了每个用户级线程能按顺序看到自己的内存访问？为什么或者为什么不呢？

想象一个大型非均匀缓存架构（NUCA）系统，为了在给定节点内为各个CPU提供公平的互连带宽分配，在每个节点的互连接口中提供了针对每颗CPU的队列，如图8所示。尽管某个特定CPU的访问顺序由该CPU执行的内存屏障所指定，但一对特定CPU之间的相对访问顺序可能会被严重重排，这一点我们将在后续内容中看到。

6.2 Example 1

图2展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。“a”、“b”和“c”初始值均为零。

⁵Any real hardware architect or designer will no doubt be loudly calling for Ralph on the porcelain intercom, as they just might be just a bit upset about the prospect of working out which queue should handle a message involving a cache line that both CPUs accessed, to say nothing of the many races that this example poses. All I can say is “Give me a better example”.

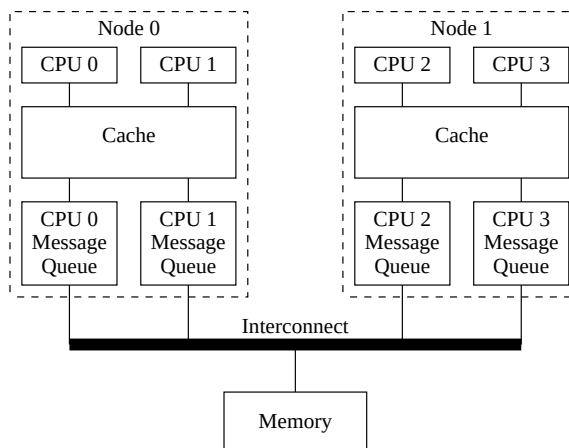


图8：示例敌序架构

假设CPU 0最近经历了许多缓存未命中，因此其消息队列已满，而CPU 1一直在缓存中运行，所以它的消息队列是空的。那么，CPU 0对“a”和“b”的赋值会立即出现在节点0的缓存中（从而对CPU 1可见），但会被阻塞在CPU 0之前的流量之后。相比之下，CPU 1对“c”的赋值将通过其之前为空的队列顺利传递。因此，CPU 2可能会先看到CPU 1对“c”的赋值，而看不到CPU 0对“a”的赋值，从而触发断言，尽管存在内存屏障。

在理论上，可移植代码无法使用这种编码技术，然而，在实际应用中，它确实可以在所有主流计算机系统上运行。

Quick Quiz 7: 在这段代码中，通过在CPU 1的“while”循环和对变量“c”的赋值之间插入一个内存屏障能否解决问题？为什么可以或不可以？

6.3 Example 2

图3展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。“a”和“b”初始值均为零。

再次假设CPU 0最近经历了许多缓存未命中，因此其消息队列已满，而CPU 1则一直独立运行在区域内。

CPU 0	CPU 1	CPU 2
a=1;	while(a==0); smp_mb(); b=1;	y=b; smp_rmb(); x=a; assert(y==0 x==1);

Table 3: Memory Barrier Example 2

	CPU 0	CPU 1	CPU 2
1	a=1;		
2	smb_wmb();		
3	b=1;		
4		while(b==0); smp_mb(); c=1;	while(b==0); smb_mb(); d=1;
5			
6	while(c==0);		
7	while(d==0);		
8	smp_mb();		
9	e=1;		assert(e==0 a==1);

Table 4: Memory Barrier Example 3

cache, so that its message queue is empty. Then CPU 0’s assignment to “a” and “b” will appear in Node 0’s cache immediately (and thus be visible to CPU 1), but will be blocked behind CPU 0’s prior traffic. In contrast, CPU 1’s assignment to “b” will sail through CPU 1’s previously empty queue. Therefore, CPU 2 might well see CPU 1’s assignment to “b” before it sees CPU 0’s assignment to “a”, causing the assertion to fire, despite the memory barriers.

In theory, portable code could not on this example coding technique, however, as before, in practice it actually does work on all mainstream computer systems.

6.4 Example 3

Figure4 shows three code fragments, executed concurrently by CPUs 1, 2, and 3. All variables are initially zero.

Note that neither CPU 1 nor CPU 2 can proceed to line 4 until they see CPU 0’s assignment to “b” on line 3. Once CPU 1 and 2 have executed their memory barriers on line 3, they are both guaranteed to see all assignments by CPU 0 preceding its memory barrier on line 2. Similarly, CPU 0’s memory barrier

on line 8 pairs with those of CPUs 1 and 2 on line 4, so that CPU 0 will not execute the assignment to “e” on line 9 until after its assignment to “a” is visible to both of the other CPUs. Therefore, CPU 2’s assertion on line 9 is guaranteed *not* to fire.

Quick Quiz 8: Suppose that lines 3-5 for CPUs 1 and 2 are in an interrupt handler, and that the CPU 2’s line 9 is run at process level. What changes, if any, are required to enable the code to work correctly, in other words, to prevent the assertion from firing? □

The Linux kernel’s `synchronize_rcu()` primitive uses an algorithm similar to that shown in this example.

7 Memory-Barrier Instructions For Specific CPUs

Each CPU has its own peculiar memory-barrier instructions, which can make portability a challenge, as indicated by Table 5. In fact, many software environments, including pthreads and Java, simply prohibit direct use of memory barriers, restricting the programmer to mutual-exclusion primitives that incorporate them to the extent that they are required. In the table, the first four columns indicate whether a given CPU allows the four possible combinations of loads and stores to be reordered. The next two columns indicate whether a given CPU allows loads and stores to be reordered with atomic instructions. With only six CPUs, we have five different combinations of load-store reorderings, and three of the four possible atomic-instruction reorderings.

The seventh column, dependent reads reordered, requires some explanation, which is undertaken in the following section covering Alpha CPUs. The short version is that Alpha requires memory barriers for readers as well as updaters of linked data structures. Yes, this does mean that Alpha can in effect fetch the data pointed to *before* it fetches the pointer itself, strange but true. Please see: http://www.openvms.compaq.com/wizard/wiz_2637.html if you think that I am just making this up. The benefit of this extremely weak memory model is that Alpha can

CPU 0	CPU 1	CPU 2
a=1;	while(a==0); smp_mb(); b=1;	y=b; smp_rmb(); x=a; assert(y==0 x==1);

表3：内存屏障示例2

	CPU 0	CPU 1	CPU 2
1	a=1;		
2	smb_wmb();		
3	b=1;	while(b==0); smp_mb(); c=1;	while(b==0); smb_mb(); d=1;
4			
5			
6	while(c==0);		
7	while(d==0);		
8	smp_mb();		
9	e=1;		assert(e==0 a==1);

表4：内存屏障示例3

缓存中的消息队列为空。此时，CPU 0 对变量 “a” 和 “b” 的赋值会立即出现在节点0的缓存中（因此对 CPU 1可见），但会被CPU 0之前的通信阻塞。相比之下，CPU 1 对变量 “b” 的赋值将通过其之前空的消息队列顺利传递。因此，CPU 2 可能会在看到 CPU 0 对 “a” 的赋值之前先看到 CPU 1 对 “b” 的赋值，从而触发断言，尽管存在内存屏障。

在理论上，这种编码技术不可用于便携式代码，然而实际上，它确实可以在所有主流计算机系统上运行。

6.4 Example 3

图4展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。所有变量初始值均为零。

注意，直到看到CPU 0在第3行对 “b” 的赋值，CPU 1和CPU 2都无法执行到第4行。一旦CPU 1和CPU 2在第3行执行了内存屏障指令，它们都将保证能看到所有发生在CPU 0的第2行内存屏障之前的赋值操作。同样地，CPU 0的内存屏障也确保了这一点。

在第8行与CPU 1和CPU 2在第4行的操作配对，因此CPU 0不会在它的对 “a” 的赋值可见于其他两个CPU之前执行对 “e” 的赋值操作。因此，CPU 2在第9行的断言必定会触发 *not*。

Quick Quiz 8: 假设CPU 1和CPU 2的第3至第5行代码位于中断处理程序中，而CPU 2的第9行在进程级别运行。为了使代码能够正确工作，即防止断言触发，需要做哪些修改？



Linux 内核的 `synchronize_rcu()` 原语使用了与该示例中所示类似的算法。

7 Memory-Barrier Instructions For Specific CPUs

每个CPU都有其独特的内存屏障指令，这使得跨平台移植变得颇具挑战性，如表5所示。事实上，许多软件环境，包括

第七列 “依赖读取重排序” 需要一些解释，这将在接下来讨论Alpha CPU的部分中进行说明。简而言之，Alpha要求对链式数据结构的读者和更新者都使用内存屏障。是的，这意味着Alpha确实可以在获取指向它的指针之前就获取该指针所指向的数据，虽然听起来很奇怪但这是事实。如果您认为我在编造这些内容，请参见：http://www.openvms.compaq.com/wizard/wiz_2637.html。这种极其弱化的内存模型的好处在于，Alpha可以

	Loads Reordered After Loads?	Loads Reordered After Stores?	Stores Reordered After Stores?	Stores Reordered After Loads?	Atomic Instructions Reordered With Loads?	Atomic Instructions Reordered With Stores?	Dependent Loads Reordered?	Incoherent Instruction Cache/Pipeline?
Alpha	Y	Y	Y	Y	Y	Y	Y	Y
AMD64				Y				
IA64	Y	Y	Y	Y	Y	Y		Y
(PA-RISC)	Y	Y	Y	Y				
PA-RISC CPUs								
POWER TM	Y	Y	Y	Y	Y	Y		Y
(SPARC RMO)	Y	Y	Y	Y	Y	Y		Y
(SPARC PSO)			Y	Y		Y		Y
SPARC TSO				Y				Y
x86				Y				Y
(x86 OOSTore)	Y	Y	Y	Y				Y
zSeries [®]				Y				Y

Table 5: Summary of Memory Ordering

use simpler cache hardware, which in turn permitted higher clock frequency in Alpha’s heyday.

The last column indicates whether a given CPU has a incoherent instruction cache and pipeline. Such CPUs require special instructions be executed for self-modifying code.

Parenthesized CPU names indicate modes that are architecturally allowed, but rarely used in practice.

The common “just say no” approach to memory barriers can be eminently reasonable where it applies, but there are environments, such as the Linux kernel, where direct use of memory barriers is required. Therefore, Linux provides a carefully chosen least-common-denominator set of memory-barrier primi-

tives, which are as follows:

- **smp_mb()**: “memory barrier” that orders both loads and stores. This means that loads and stores preceding the memory barrier will be committed to memory before any loads and stores following the memory barrier.
- **smp_rmb()**: “read memory barrier” that orders only loads.
- **smp_wmb()**: “write memory barrier” that orders only stores.
- **smp_read_barrier_depends()** that forces subsequent operations that depend on prior operations to be ordered. This primitive is a no-op on all platforms except Alpha.
- **mmiowb()** that forces ordering on MMIO writes that are guarded by global spinlocks. This primitive is a no-op on all platforms on which the memory barriers in spinlocks already enforce MMIO ordering. The platforms with a non-no-op **mmiowb()** definition include some (but not all) IA64, FRV, MIPS, and SH systems. This primitive is relatively new, so relatively few drivers take advantage of it.

The **smp_mb()**, **smp_rmb()**, and **smp_wmb()** primitives also force the compiler to eschew any optimizations that would have the effect of reordering memory optimizations across the barriers. The **smp_read_barrier_depends()** primitive has a similar effect, but only on Alpha CPUs.

These primitives generate code only in SMP kernels, however, each also has a UP version (**smp_mb()**, **smp_rmb()**, **smp_wmb()**, and **smp_read_barrier_depends()**, respectively) that generate a memory barrier even in UP kernels. The **smp_** versions should be used in most cases. However, these latter primitives are useful when writing drivers, because MMIO accesses must remain ordered even in UP kernels. In absence of memory-barrier instructions, both CPUs and compilers would happily rearrange these accesses, which at best would make the device act strangely, and could crash your kernel or, in some cases, even damage your hardware.

	sd aol r e l A d e r e d r o e R s d a o L	se r o l S r e l A d e r e d r o e R s d a o L	se r o l S r e l A d e r e d r o e R se r o l S	sd aol r e l A d e r e d r o e R se r o l S	sd aol h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A
Alpha	Y	Y	Y	Y	Y	Y	Y	Y	Y
AMD64				Y					
IA64	Y	Y	Y	Y	Y	Y	Y		Y
(PA-RISC)	Y	Y	Y	Y					
PA-RISC CPUs									
POWER ^{1.0}	Y	Y	Y	Y	Y	Y	Y		Y
(SPARC RMO)	Y	Y	Y	Y	Y	Y	Y		Y
(SPARC PSO)			Y	Y		Y			Y
SPARC TSO				Y					Y
x86				Y					Y
(x86 OOSTore)	Y	Y	Y	Y					Y
zSeries ⁶⁴				Y					Y

表5：内存顺序概要

使用更简单的缓存硬件，这反过来使得在Alpha架构的鼎盛时期能够实现更高的时钟频率。

最后一列表明给定的CPU是否具有不一致的指令缓存和流水线。此类CPU在执行自修改代码时需要特殊指令。

括号中的CPU名称表示在架构上允许但实际很少使用的模式。

对于内存屏障采取的“直接拒绝”方法在某些情况下是完全合理的，但在诸如Linux内核这样的环境中，则需要直接使用内存屏障。因此，Linux提供了一套精心挑选的、最低公倍数级别的内存屏障原语。

以下为翻译后的最终文本：目标如下：

- `smp_mb()` “内存屏障”，它规定了加载和存储操作的顺序。这意味着，在内存屏障之前的加载和存储操作将会在任何后续的加载和存储操作之前被提交到内存中。
- `smp_rmb()` “仅排序加载操作的读内存屏障”。
- `smp_wmb()` “仅排序存储操作的写内存屏障”。
- `smp_read_barrier_depends()` 这迫使依赖于先前操作的后续操作按顺序执行。此原语仅在Alpha平台上不是空操作。
- `mmiowb()` 这会强制对受全局自旋锁保护的MMIO写操作进行排序。在所有平台上的内存屏障已通过自旋锁强制执行MMIO顺序的情况下，此原语为无操作。具有非无操作定义`mmiowb()`的平台包括某些（但并非全部）IA64、FRV、MIPS和SH系统。由于该原语相对较新，因此利用它的驱动程序相对较少。

`smp_mb()`、`smp_rmb()` 和 `smp_wmb()` 原语也迫使编译器避免采用任何会重新排序内存访问优化的优化策略，这些优化可能会跨越屏障。类似地，`smp_read_barrier_depends()` 原语在Alpha CPU上也有相同的效果。

这些原语仅在SMP内核中生成代码，然而每个也有一个单处理器版本（`smp_mb()`、`tt_smp_rmb()`、`smp_wmb()`和`smp_read_barrier_depends()`），即使在UP内核中也会生成内存屏障。通常应使用`smp`版本。然而，在编写驱动程序时，这些后者的原语非常有用，因为MMIO访问必须保持有序，即便是在UP内核中也是如此。如果没有内存屏障指令，CPU和编译器都会乐意重新排列这些访问操作，这至少会导致设备行为异常，并且可能会导致你的内核崩溃，甚至在某些情况下损坏硬件。

So most kernel programmers need not worry about the memory-barrier peculiarities of each and every CPU, as long as they stick to these interfaces. If you are working deep in a given CPU's architecture-specific code, of course, all bets are off.

Furthermore, all of Linux's locking primitives (spinlocks, reader-writer locks, semaphores, RCU, ...) include any needed barrier primitives. So if you are working with code that uses these primitives, you don't even need to worry about Linux's memory-ordering primitives.

That said, deep knowledge of each CPU's memory-consistency model can be very helpful when debugging, to say nothing of when writing architecture-specific code or synchronization primitives.

Besides, they say that a little knowledge is a very dangerous thing. Just imagine the damage you could do with a lot of knowledge! For those who wish to understand more about individual CPUs' memory consistency models, the next sections describes those of the most popular and prominent CPUs. Although nothing can replace actually reading a given CPU's documentation, these sections give a good overview.

7.1 Alpha

It may seem strange to say much of anything about a CPU whose end of life has been announced, but Alpha is interesting because, with the weakest memory ordering model, it reorders memory operations the most aggressively. It therefore has defined the Linux-kernel memory-ordering primitives, which must work on all CPUs, including Alpha. Understanding Alpha is therefore surprisingly important to the Linux kernel hacker.

The difference between Alpha and the other CPUs is illustrated by the code shown in Figure 9. This `smp_wmb()` on line 9 of this figure guarantees that the element initialization in lines 6-8 is executed before the element is added to the list on line 10, so that the lock-free search will work correctly. That is, it makes this guarantee on all CPUs *except* Alpha.

Alpha has extremely weak memory ordering such that the code on line 20 of Figure 9 could see the old garbage values that were present before the initialization on lines 6-8.

```

1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         /* BUG ON ALPHA!!! */
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }

```

Figure 9: Insert and Lock-Free Search

Figure 10 shows how this can happen on an aggressively parallel machine with partitioned caches, so that alternating caches lines are processed by the different partitions of the caches. Assume that the list header `head` will be processed by cache bank 0, and that the new element will be processed by cache bank 1. On Alpha, the `smp_wmb()` will guarantee that the cache invalidates performed by lines 6-8 of Figure 9 will reach the interconnect before that of line 10 does, but makes absolutely no guarantee about the order in which the new values will reach the reading CPU's core. For example, it is possible that the reading CPU's cache bank 1 is very busy, but cache bank 0 is idle. This could result in the cache invalidates for the new element being delayed, so that the reading CPU gets the new value for the pointer, but sees the old cached values for the new element. See the Web site called out earlier for more information, or, again, if you think that I am just making all this up.⁶

One could place an `smp_rmb()` primitive be-

⁶Of course, the astute reader will have already recognized that Alpha is nowhere near as mean and nasty as it could be, the (thankfully) mythical architecture in Section 6.1 being a case in point.

因此，大多数内核程序员无需担心每一种CPU特有的内存屏障细节，只要他们遵守这些接口即可。当然，如果你在特定CPU架构的底层代码中工作，则另当别论。

此外，Linux的所有锁原语（自旋锁、读写锁、信号量、RCU等）都包含了所需的屏障原语。因此，如果你正在处理使用这些原语的代码，则无需担心Linux的内存顺序原语。

话说，深入了解每个CPU的内存一致性模型在调试时非常有帮助，更不用说编写特定架构的代码或同步原语时。

此外，他们说一点知识是非常危险的。试想一下，拥有大量知识会造成多大的破坏！对于那些希望更深入理解个别CPU内存一致性模型的人来说，接下来的部分将描述最流行和重要的CPU的相关内容。虽然没有什么能替代实际阅读特定CPU的文档，但这些部分可以提供一个很好的概览。

7.1 Alpha

说太多关于一款已经宣布停产的CPU似乎有些奇怪，但Alpha处理器却很有研究价值，因为它具有最弱的内存排序模型，并且会最为激进地重排内存操作。因此，它定义了适用于所有CPU（包括Alpha）的Linux内核内存排序原语。对于Linux内核开发者来说，理解Alpha处理器的重要性因此显得出乎意料地重要。

Alpha与其它CPU之间的区别如图9所示的代码所阐明。该代码第9行的`smp_wmb()`确保了在第6至8行进行的元素初始化会在该元素于第10行被添加到列表之前执行，从而使无锁搜索能够正确工作。也就是说，在所有CPU上（除了Alpha之外），都做出了这样的保证。

Alpha的内存顺序极为弱化，因此图9中第20行的代码可能会看到初始化前（第6至第8行）存在的旧垃圾值。

```
1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         /* BUG ON ALPHA!!! */
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }
```

图9：插入与无锁查找

图10展示了一个高度并行的机器上，当缓存被分区时这种情况是如何发生的。假设列表头`head`将由缓存银行0处理，而新元素则由缓存银行1处理。在Alpha架构中，`smp_wmb()`可以保证图9中的第6至8行执行的缓存失效操作会先于第10行的操作到达互连网络，但对新值到达读取CPU核心的顺序没有任何保证。例如，有可能读取CPU的缓存银行1非常繁忙，而缓存银行0则处于空闲状态。这可能导致针对新元素的缓存失效被延迟，从而使读取CPU获取到指针的新值，但却看到的是新元素的老缓存值。请参阅之前提到的网站以获得更多信息，或者如果您认为我只是在编造这些内容，请再次查看。⁶

可以放置一个 `smp_rmb()` 原始元素来

⁶Of course, the astute reader will have already recognized that Alpha is nowhere near as mean and nasty as it could be, the (thankfully) mythical architecture in Section 6.1 being a case in point.

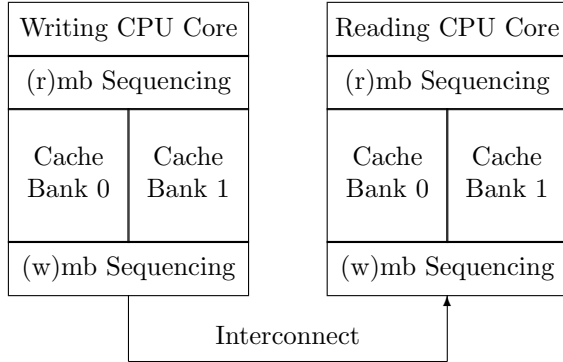


Figure 10: Why `smp_read_barrier_depends()` is Required

tween the pointer fetch and dereference. However, this imposes unneeded overhead on systems (such as i386, IA64, PPC, and SPARC) that respect data dependencies on the read side. A `smp_read_barrier_depends()` primitive has been added to the Linux 2.6 kernel to eliminate overhead on these systems. This primitive may be used as shown on line 19 of Figure 11.

It is also possible to implement a software barrier that could be used in place of `smp_wmb()`, which would force all reading CPUs to see the writing CPU's writes in order. However, this approach was deemed by the Linux community to impose excessive overhead on extremely weakly ordered CPUs such as Alpha. This software barrier could be implemented by sending inter-processor interrupts (IPIs) to all other CPUs. Upon receipt of such an IPI, a CPU would execute a memory-barrier instruction, implementing a memory-barrier shutdown. Additional logic is required to avoid deadlocks. Of course, CPUs that respect data dependencies would define such a barrier to simply be `smp_wmb()`. Perhaps this decision should be revisited in the future as Alpha fades off into the sunset.

The Linux memory-barrier primitives took their names from the Alpha instructions, so `smp_mb()` is `mb`, `smp_rmb()` is `rmb`, and `smp_wmb()` is `wmb`. Alpha is the

```

1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         smp_read_barrier_depends();
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }

```

Figure 11: Safe Insert and Lock-Free Search

only CPU where `smp_read_barrier_depends()` is an `smp_mb()` rather than a no-op.

For more detail on Alpha, see the reference manual [19].

7.2 AMD64

AMD64 is compatible with x86, and has recently updated its memory model [2] to enforce the tighter ordering that actual implementations have provided for some time. The AMD64 implementation of the Linux `smp_mb()` primitive is `mfence`, `smp_rmb()` is `lfence`, and `smp_wmb()` is `sfence`. In theory, these might be relaxed, but any such relaxation must take SSE and 3DNow instructions into account.

7.3 IA64

IA64 offers a weak consistency model, so that in absence of explicit memory-barrier instructions, IA64 is within its rights to arbitrarily reorder memory references [8]. IA64 has a memory-fence instruction named `mf`, but also has “half-memory fence” modifiers to loads, stores, and to some of its atomic

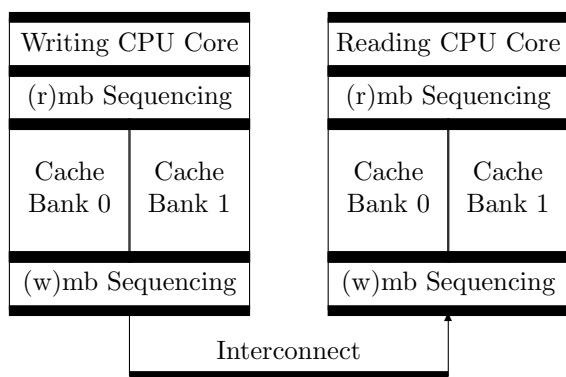


图10: 为什么SMP读屏障依赖关系是必要的

在指针获取和解引用之间。然而，这给了一些系统（如i386、IA64、PPC和SPARC）带来了不必要的开销，这些系统在读取侧会尊重数据依赖性。Linux 2.6内核中添加了一个`smp_read_barrier_depends()`原语来消除这些系统的开销。此原语可以在图11的第19行中看到如何使用。

也可以实现一种软件屏障，用以替代`smp_wmb()`，强制所有读取CPU按顺序看到写入CPU的写操作。然而，Linux社区认为这种方法在Alpha等极弱有序的CPU上会带来过高的开销。这种软件屏障可以通过向其他所有CPU发送跨处理器中断（IPI）来实现。当收到这样的IPI时，一个CPU将执行内存屏障指令，实施内存屏障清理操作。为了避免死锁，还需要额外的逻辑处理。当然，尊重数据依赖关系的CPU会将此类屏障简单定义为`smp_wmb()`。随着Alpha逐渐退出历史舞台，或许未来应重新考虑这一决定。

Linux的内存屏障原语名称源自Alpha指令，因此`smp_mb()`是`{smp_mb()}`，`smp_rmb()`是`{smp_rmb()}`，而`smp_wmb()`是`{smp_wmb()}`。Alpha架构中

```

1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         smp_read_barrier_depends();
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }

```

图11: 安全插入与锁-free搜索

仅在`smp_read_barrier_depends()`是一个`smp_mb()`而非空操作的CPU中。

参见文献[19]以获取更多关于Alpha的详细信息。

7.2 AMD64

AMD64与x86兼容，并已最近更新了其内存模型[2]，以强制执行实际实现长期以来提供的更严格的顺序。Linux在AMD64上的`smp_mb()`原语的实现为`mfence`，`smp_rmb()`为`lfence`，而`smp_wmb()`为`sfence`。理论上这些可能被放宽，但任何此类放宽都必须考虑SSE和3DNow指令的影响。

7.3 IA64

IA64提供了一种弱一致性模型，因此在没有显式内存屏障指令的情况下，IA64有权任意重排内存引用[8]。IA64有一个名为`mf`的内存屏障指令，但也为加载、存储以及某些原子操作提供了“半内存屏障”修饰符。



Figure 12: Half Memory Barrier

instructions [7]. The `acq` modifier prevents subsequent memory-reference instructions from being reordered before the `acq`, but permits prior memory-reference instructions to be reordered after the `acq`, as fancifully illustrated by Figure 12. Similarly, the `rel` modifier prevents prior memory-reference instructions from being reordered after the `rel`, but allows subsequent memory-reference instructions to be reordered before the `rel`.

These half-memory fences are useful for critical sections, since it is safe to push operations into a critical section, but can be fatal to allow them to bleed out. However, as one of the only CPUs with this property, IA64 defines Linux’s semantics of memory ordering associated with lock acquisition and release.

The IA64 `mf` instruction is used for the `smp_rmb()`, `smp_mb()`, and `smp_wmb()` primitives in the Linux kernel. Oh, and despite rumors to the contrary, the “mf” mnemonic really does stand for “memory fence”.

Finally, IA64 offers a global total order for “release” operations, including the “mf” instruction. This provides the notion of transitivity, where if a given code fragment sees a given access as having happened, any later code fragment will also see that earlier access as having happened. Assuming, that is, that all the code fragments involved correctly use memory barriers.

7.4 PA-RISC

Although the PA-RISC architecture permits full re-ordering of loads and stores, actual CPUs run fully ordered [14]. This means that the Linux kernel’s memory-ordering primitives generate no code, however, they do use the `gcc memory` attribute to disable compiler optimizations that would reorder code across the memory barrier.

7.5 POWER

The POWER and Power PC® CPU families have a wide variety of memory-barrier instructions [6, 15]:

1. `sync` causes all preceding operations to *appear to have* completed before any subsequent operations are started. This instruction is therefore quite expensive.
2. `lwsync` (light-weight sync) orders loads with respect to subsequent loads and stores, and also orders stores. However, it does *not* order stores with respect to subsequent loads. Interestingly enough, the `lwsync` instruction enforces the same ordering as does zSeries, and coincidentally, SPARC TSO.
3. `eieio` (enforce in-order execution of I/O, in case you were wondering) causes all preceding cacheable stores to appear to have completed before all subsequent stores. However, stores to cacheable memory are ordered separately from stores to non-cacheable memory, which means that `eieio` will not force an MMIO store to precede a spinlock release.
4. `isync` forces all preceding instructions to appear to have completed before any subsequent instructions start execution. This means that the preceding instructions must have progressed far enough that any traps they might generate have either happened or are guaranteed not to happen, and that any side-effects of these instructions (for example, page-table changes) are seen by the subsequent instructions.

Unfortunately, none of these instructions line up exactly with Linux’s `wmb()` primitive, which requires



图12：半内存屏障

指令[7]。acq修饰符防止后续的内存引用指令被重新排序到acq之前，但允许之前的内存引用指令在acq之后进行重排。类似地，rel修饰符防止之前的内存引用指令被重新排序到rel之后，但允许后续的内存引用指令在rel之前进行重排。

这些半内存栅栏在临界区中非常有用，因为可以安全地将操作推进到临界区内，但允许它们溢出则可能致命。然而，作为少数具有此特性的CPU之一，IA64定义了Linux中与锁获取和释放相关的内存顺序语义。

IA64 v6 指令用于 Linux 内核中的 v7、v8 和 v9 原语。哦，尽管有传言称不然，“mf”助记符确实代表“内存栅栏”。

最终，IA64为“释放”操作提供了全局总顺序，包括“mf”指令。这提供了一种传递性的概念，即如果一段给定的代码片段看到某个访问已经发生，则任何后续的代码片段也将看到该早期访问已发生。假设所有涉及的代码片段都正确使用了内存屏障。

7.4 PA-RISC

尽管PA-RISC架构允许完全重新排序加载和存储操作，但实际上的CPU是全序执行的[14]。这意味着Linux内核中的内存顺序原语不会生成任何代码，然而它们确实使用了gcc memory属性来禁用编译器优化，这些优化可能会在内存屏障两侧重排代码。

7.5 POWER

POWER 和 PowerPC® 系列处理器具有多种内存屏障指令 [6, 15]:

sync 导致所有先前的操作必须完全完成，才能开始任何后续操作。因此，这条指令相当昂贵。

轻量级同步指令会对后续的加载和存储操作进行排序，同时也对存储操作进行排序。然而，它不会对后续的加载操作与当前的存储操作之间进行排序。有趣的是，该指令执行的排序规则与zSeries系统以及SPARC TSO一致。

3. 如你所想，eieio (enforce in-order execution of I/O)会确保所有之前的可缓存存储操作在所有后续的存储操作之前完成。然而，对可缓存内存的存储操作与对不可缓存内存的存储操作是分别排序的，这意味着eieio不会强制一个MMIO存储操作先于自旋锁释放操作执行。

4. isync强制所有先前的指令在任何后续指令开始执行之前必须看起来已经完成。这意味着之前的指令必须足够进展，以确保它们可能引发的所有异常（如陷阱）要么已经发生，要么可以保证不会发生，并且这些指令所产生的所有副作用（例如页表更改）会被后续指令观察到。

不幸的是，这些指令与Linux的wmb()原语并不完全对应，后者要求

all stores to be ordered, but does not require the other high-overhead actions of the `sync` instruction. But there is no choice: ppc64 versions of `wmb()` and `mb()` are defined to be the heavyweight `sync` instruction. However, Linux’s `smp_wmb()` instruction is never used for MMIO (since a driver must carefully order MMIOs in UP as well as SMP kernels, after all), so it is defined to be the lighter weight `eieio` instruction. This instruction may well be unique in having a five-vowel mnemonic, which stands for “enforce in-order execution of I/O”. The `smp_mb()` instruction is also defined to be the `sync` instruction, but both `smp_rmb()` and `rmb()` are defined to be the lighter-weight `lwsync` instruction.

Power features “cumulativity”, which can be used to obtain transitivity. When used properly, any code seeing the results of an earlier code fragment will also see the accesses that this earlier code fragment itself saw. Much more detail is available from McKenney and Silvera [18].

Many members of the POWER architecture have incoherent instruction caches, so that a store to memory will not necessarily be reflected in the instruction cache. Thankfully, few people write self-modifying code these days, but JITs and compilers do it all the time. Furthermore, recompiling a recently run program looks just like self-modifying code from the CPU’s viewpoint. The `icbi` instruction (instruction cache block invalidate) invalidates a specified cache line from the instruction cache, and may be used in these situations.

7.6 SPARC RMO, PSO, and TSO

Solaris on SPARC uses TSO (total-store order), as does Linux when built for the “sparc” 32-bit architecture. However, a 64-bit Linux kernel (the “sparc64” architecture) runs SPARC in RMO (relaxed-memory order) mode [20]. The SPARC architecture also offers an intermediate PSO (partial store order). Any program that runs in RMO will also run in either PSO or TSO, and similarly, a program that runs in PSO will also run in TSO. Moving a shared-memory parallel program in the other direction may require careful insertion of memory barriers, although, as noted earlier, programs that make standard use of synchroniza-

tion primitives need not worry about memory barriers.

SPARC has a very flexible memory-barrier instruction [20] that permits fine-grained control of ordering:

StoreStore: order preceding stores before subsequent stores. (This option is used by the Linux `smp_wmb()` primitive.)

LoadStore: order preceding loads before subsequent stores.

StoreLoad: order preceding stores before subsequent loads.

LoadLoad: order preceding loads before subsequent loads. (This option is used by the Linux `smp_rmb()` primitive.)

Sync: fully complete all preceding operations before starting any subsequent operations.

MemIssue: complete preceding memory operations before subsequent memory operations, important for some instances of memory-mapped I/O.

Lookaside: same as `MemIssue`, but only applies to preceding stores and subsequent loads, and even then only for stores and loads that access the same memory location.

The Linux `smp_mb()` primitive uses the first four options together, as in `membar #LoadLoad | #LoadStore | #StoreStore | #StoreLoad`, thus fully ordering memory operations.

So, why is `membar #MemIssue` needed? Because a `membar #StoreLoad` could permit a subsequent load to get its value from a write buffer, which would be disastrous if the write was to an MMIO register that induced side effects on the value to be read. In contrast, `membar #MemIssue` would wait until the write buffers were flushed before permitting the loads to execute, thereby ensuring that the load actually gets its value from the MMIO register. Drivers could instead use `membar #Sync`, but the lighter-weight `membar #MemIssue` is preferred in cases where the additional function of the more-expensive `membar #Sync` are not required.

all 存储操作需要有序执行，但不需要其他高开销的操作。然而别无选择：ppc64版本的wmb()和mb()被定义为重量级的sync指令。但是，Linux中的smp_wmb()指令从未用于MMIO（毕竟，在UP和SMP内核中，驱动程序必须仔细地MMIO进行排序），因此它被定义为轻量级的eieio指令。这条指令可能是唯一一个具有五个元音字母助记符的指令，其含义是“强制I/O按顺序执行”。同样，smp_mb()指令也被定义为sync指令，但smp_rmb()和rmb()都被定义为轻量级的lwsync指令。

功率特性具有“累积性”，可以用来获得传递性。当使用得当时，任何看到早期代码片段结果的代码也将看到该早期代码片段本身访问的内容。更多细节可参见 McKenney 和 Silvera [18]。

许多POWER架构的成员具有非一致性的指令缓存，因此对内存中的存储操作不一定反映在指令缓存中。值得庆幸的是，如今很少有人编写自修改代码，但JIT编译器和传统编译器却经常这样做。此外，重新编译最近运行过的程序从CPU的角度来看就像自修改代码一样。icbi指令（指令缓存块无效化）会将指定的缓存行从指令缓存中失效，并且可以在这些情况下使用该指令。

7.6 SPARC RMO, PSO, and TSO

Solaris在SPARC架构上使用TSO（全存.Store顺序），而当Linux为“sparc”32位架构构建时也是如此。然而，64位的Linux内核（即“sparc64”架构）则以RMO（宽松内存顺序）模式运行SPARC [20]。SPARC架构还提供了一种中间级PSO（部分存.Store顺序）。任何在RMO下运行的程序也将在PSO或TSO下运行，同样地，任何在PSO下运行的程序也将能在TSO下运行。然而，将一个共享内存并行程序向相反方向移动可能需要谨慎插入内存屏障，尽管如前所述，标准使用同步机制的程序可以避免这一问题。

内存操作原语无需担心内存屏障。

SPARC 提供了一个非常灵活的内存屏障指令[20]，允许对顺序进行细粒度控制：

StoreStore: 将前面的存储指令排序在随后的存储指令之前。（此选项由Linux smp_wmb()原语使用。）

LoadStore: 将前序的加载指令排序在后续的存储指令之前。

StoreLoad: 将前面的存储指令排序在后续的加载指令之前。

LoadLoad: 将先前的加载指令排序在随后的加载指令之前。（此选项由Linux smp_rmb()原语使用。）

Sync完成所有先前的操作后再开始后续的操作。

MemIssue完成先前的内存操作后再进行后续的内存操作，这对于某些内存映射I/O的情况非常重要。

Lookaside与MemIssue相同，但仅适用于先前的存储操作和后续的加载操作，并且仅针对访问同一内存位置的存储和加载操作。

Linux smp_mb()使用前四个选项的组合，如membar #LoadLoad | #LoadStore | #StoreStore | #StoreLoad所示，从而完全有序地执行内存操作。

因此，为什么需要membar #MemIssue? 因为一个membar #StoreLoad可能会允许后续的加载操作从写缓冲区获取值，如果这次写入的是具有副作用的MMIO寄存器，则这将是非常灾难性的。相比之下，membar #MemIssue会在允许加载操作执行之前等待写缓冲区被刷新，从而确保加载操作确实是从MMIO寄存器中获取值。驱动程序也可以使用membar #Sync，但在不需要更昂贵的membar #Sync所提供的额外功能的情况下，较轻量级的membar #MemIssue更为优选。

The `membar #Lookaside` is a lighter-weight version of `membar #MemIssue`, which is useful when writing to a given MMIO register affects the value that will next be read from that register. However, the heavier-weight `membar #MemIssue` must be used when a write to a given MMIO register affects the value that will next be read from *some other* MMIO register.

It is not clear why SPARC does not define `wmb()` to be `membar #MemIssue` and `smb_wmb()` to be `membar #StoreStore`, as the current definitions seem vulnerable to bugs in some drivers. It is quite possible that all the SPARC CPUs that Linux runs on implement a more conservative memory-ordering model than the architecture would permit.

SPARC requires a `flush` instruction be used between the time that an instruction is stored and executed [20]. This is needed to flush any prior value for that location from the SPARC's instruction cache. Note that `flush` takes an address, and will flush only that address from the instruction cache. On SMP systems, all CPUs' caches are flushed, but there is no convenient way to determine when the off-CPU flushes complete, though there is a reference to an implementation note.

7.7 x86

Since the x86 CPUs provide “process ordering” so that all CPUs agree on the order of a given CPU's writes to memory, the `smp_wmb()` primitive is a no-op for the CPU [10]. However, a compiler directive is required to prevent the compiler from performing optimizations that would result in reordering across the `smp_wmb()` primitive.

On the other hand, x86 CPUs have traditionally given no ordering guarantees for loads, so the `smp_mb()` and `smp_rmb()` primitives expand to `lock;addl`. This atomic instruction acts as a barrier to both loads and stores.

More recently, Intel has published a memory model for x86 [11]. It turns out that Intel's actual CPUs enforced tighter ordering than was claimed in the previous specifications, so this model is in effect simply mandating the earlier de-facto behavior. Even more recently, Intel published an updated memory model

for x86 [12], which mandates a total global order for stores, although individual CPUs are still permitted to see their own stores as having happened earlier than this total global order would indicate. This exception to the total ordering is needed to allow important hardware optimizations involving store buffers. Software may use atomic operations to override these hardware optimizations, which is one reason that atomic operations tend to be more expensive than their non-atomic counterparts. This total store order is *not* guaranteed on older processors.

However, note that some SSE instructions are weakly ordered (`clflush` and non-temporal move instructions [9]). CPUs that have SSE can use `mfence` for `smp_mb()`, `lfence` for `smp_rmb()`, and `sfence` for `smp_wmb()`.

A few versions of the x86 CPU have a mode bit that enables out-of-order stores, and for these CPUs, `smp_wmb()` must also be defined to be `lock;addl`.

Although many older x86 implementations accommodated self-modifying code without the need for any special instructions, newer revisions of the x86 architecture no longer requires x86 CPUs to be so accommodating. Interestingly enough, this relaxation comes just in time to inconvenience JIT implementors.

7.8 zSeries

The zSeries machines make up the IBM[™] mainframe family, previously known as the 360, 370, and 390 [13]. Parallelism came late to zSeries, but given that these mainframes first shipped in the mid 1960s, this is not saying much. The `bcr 15,0` instruction is used for the Linux `smp_mb()`, `smp_rmb()`, and `smp_wmb()` primitives. It also has comparatively strong memory-ordering semantics, as shown in Table 5, which should allow the `smp_wmb()` primitive to be a `nop` (and by the time you read this, this change may well have happened). The table actually understates the situation, as the zSeries memory model is otherwise sequentially consistent, meaning that all CPUs will agree on the order of unrelated stores from different CPUs.

As with most CPUs, the zSeries architecture does not guarantee a cache-coherent instruction stream,

`membar #Lookaside` 是 `membar #MemIssue` 的轻量级版本，适用于向某个 MMIO 寄存器写入数据会直接影响该寄存器下次读取值的场景。然而，当对某个 MMIO 寄存器的写操作会影响另一个 *some other* MMIO 寄存器的下次读取值时，则必须使用更 *heavy weight* 的 `membar #MemIssue`。

不清楚为什么 SPARC 没有将 `wmb()` 定义为 `membar #MemIssue`，将 `smp_wmb()` 定义为 `membar #StoreStore`，因为当前的定义似乎容易受到某些驱动程序中错误的影响。很有可能所有运行 Linux 的 SPARC CPU 都实现了一种比架构允许的更为保守的内存排序模型。

SPARC 要求在指令被存储和执行之间使用一条 `flush` 指令[20]。这是为了从 SPARC 的指令缓存中清除该位置之前的任何值。注意，`flush` 需要一个地址，并且只会从指令缓存中清除指定的那个地址。在 SMP 系统上，所有 CPU 的缓存都会被刷新，但没有方便的方法来确定跨 CPU 刷新何时完成，尽管有一份实现说明提到这一点。

7.7 x86

由于 x86 处理器提供了“进程顺序”，确保所有 CPU 对特定 CPU 写入内存的操作顺序达成一致，因此 `smp_wmb()` 原语对于 CPU 而言是无操作的[10]。然而，需要编译器指令来防止编译器执行可能导致跨 `smp_wmb()` 原语重新排序的优化操作。

另一方面，x86 CPU 传统上对加载操作不提供任何顺序保证，因此 `smp_mb()` 和 `smp_rmb()` 原语展开为 `lock;addl`。这条原子指令既作为加载操作的屏障也作为存储操作的屏障。

最近，英特尔为 x86 架构发布了一套内存模型 [11]。结果发现，英特尔的实际 CPU 执行的顺序约束比之前规范中声明的更为严格，因此这套模型实际上只是强制规定了早期的事实上的行为。更近一些时候，英特尔发布了更新后的内存模型。

对于 x86 架构[12]，虽然要求全局存储操作遵循总序规则，但单个 CPU 仍可将其自身的存储操作视为早于该总序规则所指示的时间点发生。这一对总序的例外是为了允许重要的涉及存储缓冲区的硬件优化。软件可以使用原子操作来覆盖这些硬件优化，这也是为什么原子操作通常比非原子操作更昂贵的原因之一。这种全局存储顺序在较旧的处理器上是受 *not* 保证的。

然而，需要注意的是，某些 SSE 指令是弱有序的（`clflush` 和非临时移动指令 [9]）。具有 SSE 功能的 CPU 可以使用 `mfence` 来实现 `smp_mb()`，使用 `lfence` 来实现 `smp_rmb()`，以及使用 `sfence` 来实现 `smp_wmb()`。

某些版本的 x86 处理器具有一个模式位，用于启用乱序存储，对于这些 CPU，`smp_wmb()` 也必须被定义为 `lock;addl`。

尽管许多较早的 x86 实现能够支持自修改代码而无需任何特殊指令，但新版本的 x86 架构不再要求 x86 CPU 必须如此宽容。有趣的是，这种放宽正好赶上了让即时编译器实现者感到不便的时候。

7.8 zSeries

zSeries 机器构成了 IBM 的大型机家族，此前被称为 360、370 和 390 系列[13]。尽管并行处理在 zSeries 中出现较晚，但考虑到这些大型机早在 20 世纪 60 年代中期就开始发货，这并不算什么。该指令用于 Linux 的原语，并且具有相对较强的记忆顺序语义，如表 5 所示，这应该允许原语成为一个，并且到您阅读本文时，这一变化可能已经发生。实际上，表格低估了情况，因为 zSeries 内存模型在其他情况下是顺序一致的，这意味着所有 CPU 都将同意来自不同 CPU 的无关存储指令的顺序。

与大多数 CPU 一样，zSeries 架构不保证缓存一致的指令流。

hence, self-modifying code must execute a serializing instruction between updating the instructions and executing them. That said, many actual zSeries machines do in fact accommodate self-modifying code without serializing instructions. The zSeries instruction set provides a large set of serializing instructions, including compare-and-swap, some types of branches (for example, the aforementioned `bcr 15,0` instruction), and test-and-set, among others.

8 Are Memory Barriers Forever?

There have been a number of recent systems that are significantly less aggressive about out-of-order execution in general and re-ordering memory references in particular. Will this trend continue to the point where memory barriers are a thing of the past?

The argument in favor would cite proposed massively multi-threaded hardware architectures, so that each thread would wait until memory was ready, with tens, hundreds, or even thousands of other threads making progress in the meantime. In such an architecture, there would be no need for memory barriers, because a given thread would simply wait for all outstanding operations to complete before proceeding to the next instruction. Because there would be potentially thousands of other threads, the CPU would be completely utilized, so no CPU time would be wasted.

The argument against would cite the extremely limited number of applications capable of scaling up to a thousand threads, as well as increasingly severe realtime requirements, which are in the tens of microseconds for some applications. The realtime-response requirements are difficult enough to meet as is, and would be even more difficult to meet given the extremely low single-threaded throughput implied by the massive multi-threaded scenarios.

Another argument in favor would cite increasingly sophisticated latency-hiding hardware implementation techniques that might well allow the CPU to provide the illusion of fully sequentially consistent execution while still providing almost all of the performance advantages of out-of-order execution.

A counter-argument would cite the increasingly severe power-efficiency requirements presented both by battery-operated devices and by environmental responsibility.

Who is right? We have no clue, so are preparing to live with either scenario.

9 Advice to Hardware Designers

There are a number of things that hardware designers can do to make the lives of software people difficult. Here is a list of a few such things that we have encountered in the past, presented here in the hope that it might help prevent future such problems:

1. I/O devices that ignore cache coherence.

This charming misfeature can result in DMAs from memory missing recent changes to the output buffer, or, just as bad, cause input buffers to be overwritten by the contents of CPU caches just after the DMA completes. To make your system work in face of such misbehavior, you must carefully flush the CPU caches of any location in any DMA buffer before presenting that buffer to the I/O device. And even then, you need to be *very* careful to avoid pointer bugs, as even a misplaced read to an input buffer can result in corrupting the data input!

2. Device interrupts that ignore cache coherence.

This might sound innocent enough — after all, interrupts aren't memory references, are they? But imagine a CPU with a split cache, one bank of which is extremely busy, therefore holding onto the last cacheline of the input buffer. If the corresponding I/O-complete interrupt reaches this CPU, then that CPU's memory reference to the last cache line of the buffer could return old data, again resulting in data corruption, but in a form that will be invisible in a later crash dump. By the time the system gets around to dumping the offending input buffer, the DMA will most likely have completed.

因此，自修改代码必须在更新指令和执行它们之间插入一条串行化指令。不过，许多实际的zSeries机器确实可以在没有串行化指令的情况下支持自修改代码。zSeries指令集提供了一组大量的串行化指令，包括比较并交换、某些类型的分支（例如前述的bcr 15,0指令）以及测试并设置等。

8 Are Memory Barriers Forever?

近期出现了一些系统，在整体上对乱序执行不那么激进，特别是在重新排序内存引用方面尤为明显。这一趋势是否会继续发展到内存屏障成为过去式的地步？

支持这一观点的论据会提到拟议中的大规模多线程硬件架构，在这种架构中，每个线程都会等待内存准备好，而在其间，其他数十、数百甚至数千个线程可以继续推进。在这种架构下，无需使用内存屏障，因为一个特定的线程只需在所有未完成的操作完成后才执行下一个指令。由于可能有成千上万个其他线程存在，CPU将被完全利用，因此不会浪费任何CPU时间。

反对的观点认为，能够扩展到一千个线程的应用程序数量极为有限，并且实时需求越来越严格，某些应用的实时响应要求已经达到数十微秒。现有的实时响应要求已经很难满足，而在大规模多线程场景下，单线程吞吐量极低的情况下，这一问题将变得更加严峻。

另一个论点认为，日益复杂的延迟隐藏硬件实现技术可能会使CPU在提供几乎全部乱序执行的性能优势的同时，还能提供完全顺序一致性的执行幻象。

一种反对观点认为，无论是由电池供电的设备还是环境保护责任都提出了越来越严格的能量效率要求。

谁对谁错我们无从得知，因此正在为两种可能的情况做准备。

9 Advice to Hardware Designers

硬件设计师可以采取多种措施来增加软件开发人员的工作难度。以下是我们在过去遇到的一些例子，希望这些经验能有助于预防未来出现类似问题：

忽略缓存一致性的I/O设备。

这种迷人的缺陷可能导致从内存发起的直接存储器访问（DMA）操作未能捕获输出缓冲区中的最近更改，或者更糟糕的是，在DMA完成之后立即将输入缓冲区的内容覆盖为CPU缓存中的内容。为了使系统在这种行为面前正常工作，您必须在将任何DMA缓冲区呈现给I/O设备之前，仔细地清空该缓冲区内所有位置的CPU缓存。即便如此，仍需小心避免指针错误，因为即使是对输入缓冲区的一次误读也可能导致数据输入损坏。

2. 忽略缓存一致性的设备中断。

这听起来似乎无伤大雅——毕竟，中断并不是内存引用，对吧？但设想一个具有分隔缓存的CPU，其中一部分非常繁忙，因此持有输入缓冲区的最后一段缓存行。如果相应的I/O完成中断到达该CPU，则该CPU对缓冲区最后一段缓存行的内存引用可能会返回旧数据，从而再次导致数据损坏，但在后续的崩溃转储中将无法察觉。等到系统开始转储有问题的输入缓冲区时，DMA很可能已经完成了。

3. Inter-processor interrupts (IPIs) that ignore cache coherence.

This can be problematic if the IPI reaches its destination before all of the cache lines in the corresponding message buffer have been committed to memory.

4. Context switches that get ahead of cache coherence.

If memory accesses can complete too wildly out of order, then context switches can be quite harrowing. If the task flits from one CPU to another before all the memory accesses visible to the source CPU make it to the destination CPU, then the task could easily see the corresponding variables revert to prior values, which can fatally confuse most algorithms.

5. Overly kind simulators and emulators.

It is difficult to write simulators or emulators that force memory re-ordering, so software that runs just fine in these environments can get a nasty surprise when it first runs on the real hardware. Unfortunately, it is still the rule that the hardware is more devious than are the simulators and emulators, but we hope that this situation changes.

Again, we encourage hardware designers to avoid these practices!

Acknowledgements

I own thanks to many CPU architects for patiently explaining the instruction- and memory-reordering features of their CPUs, particularly Wayne Cardoza, Ed Silha, Anton Blanchard, Brad Frey, Cathy May, Derek Williams, Tim Slegel, Juergen Probst, Ingo Adlung, and Ravi Arimilli. Wayne deserves special thanks for his patience in explaining Alpha's reordering of dependent loads, a lesson that I resisted quite strenuously!

Legal Statement

This work represents the view of the author and does not necessarily represent the view of IBM.

IBM, zSeries, and Power PC are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both.

Linux is a registered trademark of Linus Torvalds.

i386 is a trademark of Intel Corporation or its subsidiaries in the United States, other countries, or both.

Other company, product, and service names may be trademarks or service marks of such companies.

Copyright © 2005 by IBM Corporation.

Answers to Quick Quizzes

Quick Quiz 1:

Answer: What happens if two CPUs attempt to invalidate the same cache line concurrently? ☐

One of the CPUs gains access to the shared bus first, and that CPU “wins”. The other CPU must invalidate its copy of the cache line and transmit an “invalidate acknowledge” message to the other CPU. Of course, the losing CPU can be expected to immediately issue a “read invalidate” transaction, so the winning CPU’s victory will be quite ephemeral.

Quick Quiz 2:

Answer: When an “invalidate” message appears in a large multiprocessor, every CPU must give an “invalidate acknowledge” response. Wouldn’t the resulting “storm” of “invalidate acknowledge” responses totally saturate the system bus? ☐

It might, if large-scale multiprocessors were in fact implemented that way. Larger multiprocessors, particularly NUMA machines, tend to use so-called “directory-based” cache-coherence protocols to avoid this and other problems.

Quick Quiz 3:

Answer: If SMP machines are really using message

3. 忽略缓存一致性的进程间中断（IPIs）。

这可能会出现问題，如果中断处理器指令（IPI）在其对应消息缓冲区中的所有缓存行都未提交到内存之前就已到达目的地。

4. 超前于缓存一致性的上下文切换。

如果内存访问可以过于无序地完成，那么上下文切换可能会非常糟糕。如果任务在所有对源CPU可见的内存访问到达目标CPU之前就在一个CPU和另一个CPU之间快速切换，那么该任务可能很容易看到相应的变量恢复到之前的值，这会严重混淆大多数算法。

5. 过于友好的模拟器和仿真器。

在实际编写模拟器或仿真器以强制内存重排序时会遇到困难，因此在这种环境中运行良好的软件首次在真实硬件上运行时可能会遭遇令人不悦的状况。不幸的是，目前硬件仍然比模拟器和仿真器更加复杂多变，但我们希望这种情况能够得到改善。

再次鼓励硬件设计师避免这些做法！

Acknowledgements

我感谢许多CPU架构师耐心地解释了他们CPU中的指令和内存重排序特性，特别是Wayne Cardoza、Ed Silha、Anton Blanchard、Brad Frey、Cathy May、Derek Williams、Tim Slegel、Juergen Probst、Ingo Adlung和Ravi Arimilli。Wayne尤其值得特别感谢，因为他耐心地解释了Alpha处理器对依赖性加载的重排序机制，而我对这一课的接受过程非常艰难。

Legal Statement

本文仅代表作者观点，不代表IBM的立场。

IBM、zSeries 和 Power PC 是国际商业机器 corporation 在美国及其他国家或地区的注册商标。Linux 是 Linus Torvalds 的注册商标。i386 是 Intel Corporation 或其子公司在美国及其他国家或地区的商标。其他公司产品和服务名称可能是相应公司的商标或服务标志。© 2005 IBM Corporation。

Answers to Quick Quizzes

Quick Quiz 1:

Answer: 如果两个CPU同时尝试使同一缓存行失效会发生什么？第一个获得共享总线访问权的CPU“获胜”。另一个CPU必须使其持有的该缓存行失效，并向获胜的CPU发送一个“失效确认”消息。当然，失败的CPU会立即发出一次“读取并失效”的事务请求，因此获胜CPU的优势将是短暂的。

Quick Quiz 2:

Answer: 当一个“无效”消息出现在大型多处理器系统中时，每个CPU都必须给出一个“无效确认”响应。这种大量的“无效确认”响应是否会完全饱和系统总线？

大规模多处理器系统如果确实那样实现的话，大型多处理器系统，尤其是非统一内存访问（NUMA）机器，通常会采用所谓的“目录式”缓存一致性协议来避免此类及其他问题。

Quick Quiz 3:

Answer: 如果SMP机器确实使用消息

passing anyway, why bother with SMP at all? ☐ There has been quite a bit of controversy on this topic over the past few decades. One answer is that the cache-coherence protocols are quite simple, and therefore can be implemented directly in hardware, gaining bandwidths and latencies unattainable by software message passing. Another answer is that the real truth is to be found in economics due to the relative prices of large SMP machines and that of clusters of smaller SMP machines. A third answer is that the SMP programming model is easier to use than that of distributed systems, but a rebuttal might note the appearance of HPC clusters and MPI. And so the argument continues.

Quick Quiz 4:

Answer: How does the hardware handle the delayed transitions described above? ☐

Usually by adding additional states, though these additional states need not be actually stored with the cache line, due to the fact that only a few lines at a time will be transitioning. The need to delay transitions is but one issue that results in real-world cache coherence protocols being much more complex than the over-simplified MESI protocol described in this appendix. Hennessy and Patterson's classic introduction to computer architecture [5] covers many of these issues.

Quick Quiz 5:

Answer: What sequence of operations would put the CPUs' caches all back into the "invalid" state? ☐

There is no such sequence, at least in absence of special "flush my cache" instructions in the CPU's instruction set. Most CPUs do have such instructions.

Quick Quiz 6:

Answer: Does the guarantee that each CPU sees its own memory accesses in order also guarantee that each user-level thread will see its own memory accesses in order? Why or why not? ☐

No. Consider the case where a thread migrates from one CPU to another, and where the destination CPU perceives the source CPU's recent memory operations out of order. To preserve user-mode sanity, kernel hackers must use memory barriers in the context-switch path. However, the locking already required to safely do a context switch should automatically provide the memory barriers needed to cause the user-level task to see its own accesses in order. That said, if you are designing a super-optimized scheduler, either in the kernel or at user level, please keep this scenario in mind!

Quick Quiz 7:

Answer: Could this code be fixed by inserting a memory barrier between CPU 1's "while" and assignment to "c"? Why or why not? ☐

No. Such a memory barrier would only force ordering local to CPU 1. It would have no effect on the relative ordering of CPU 0's and CPU 1's accesses, so the assertion could still fail. However, all mainstream computer systems provide one mechanism or another to provide "transitivity", which provides intuitive causal ordering: if B saw the effects of A's accesses, and C saw the effects of B's accesses, then C must also see the effects of A's accesses.

Quick Quiz 8:

Answer: Suppose that lines 3-5 for CPUs 1 and 2 are in an interrupt handler, and that the CPU 2's line 9 is run at process level. What changes, if any, are required to enable the code to work correctly, in other words, to prevent the assertion from firing? ☐

The assertion will need to be coded so as to ensure that the load of "e" precedes that of "a". In the Linux kernel, the `barrier()` primitive may be used to accomplish this in much the same way that the memory barrier was used in the assertions in the previous examples.

References

- [1] ADVANCED MICRO DEVICES. *AMD x86-64 Ar-*

随意而论，为何还要使用SMP呢？过去几十年来，这个问题一直存在不少争议。一种回答是缓存一致性协议相对简单，因此可以直接在硬件中实现，从而获得软件消息传递无法达到的带宽和低延迟。另一种回答则是从经济角度考虑，大型SMP机器与小型SMP集群的成本差异决定了其选择。还有一种回答是SMP编程模型比分布式系统更容易使用，但反驳者可能会指出高性能计算集群和MPI的出现。因此，争论仍在继续。

Quick Quiz 4:

Answer: 硬件是如何处理上述延迟过渡的？

通常通过添加额外的状态来实现这一点，尽管这些额外状态不必实际存储在缓存行中，因为一次只有少数几行会处于过渡状态。延迟转换的需要只是导致现实世界中的缓存一致性协议比本附录描述的简化的MESI协议复杂得多的问题之一。Hennessy和Patterson的经典计算机体系结构导论[5]涵盖了这些问题中的许多。

Quick Quiz 5:

Answer: 什么样的操作序列可以将所有CPU缓存都置回“无效”状态？

至少在CPU指令集中不存在特殊的“清空缓存”指令的情况下，这样的序列并不存在。不过大多数CPU确实拥有此类指令。

Quick Quiz 6:

Answer: 每个CPU都能按顺序看到自己的内存访问，这是否也保证了每个用户级线程能按顺序看到自己的内存访问？为什么或者为什么不呢？

考虑线程从一个CPU迁移到另一个CPU的情况，目标CPU感知到源CPU最近的内存操作出现乱序。为了保持用户模式下的正常状态，内核开发者必须在线程切换路径中使用内存屏障。然而，已经要求的锁定机制应该自动提供所需的内存屏障，以确保用户级任务能够按顺序看到自己的访问操作。也就是说，在设计超级优化调度器时（无论是内核层面还是用户层面），请务必牢记这一场景！

Quick Quiz 7:

Answer: 这段代码能否通过在CPU 1的“while”循环和对变量“c”的赋值之间插入一个内存屏障来修复？为什么不能？不可以。这样的内存屏障只会强制本地于CPU 1的顺序，不会影响CPU 0与CPU 1访问之间的相对顺序，因此断言仍然可能失败。然而，所有主流计算机系统都提供某种机制以实现“传递性”，即直观的因果顺序：如果B看到了A访问的效果，并且C看到了B访问的效果，则C也必须看到A访问的效果。

Quick Quiz 8:

Answer: 假设CPU 1和CPU 2的第3至5行代码处于中断处理程序中，而CPU 2的第9行在进程级别运行。为了使代码正确工作，即防止断言触发，需要做哪些修改？断言必须编写为确保“e”的加载先于“a”的加载进行。在Linux内核中，可以使用barrier()原语以与之前示例中的内存屏障类似的方式实现这一点。

References

【v11】。【v12】

- chitecture Programmer's Manual Volumes 1-5*, 2002.
- [2] ADVANCED MICRO DEVICES. *AMD x86-64 Architecture Programmer's Manual Volume 2: System Programming*, 2007.
 - [3] CULLER, D. E., SINGH, J. P., AND GUPTA, A. *Parallel Computer Architecture: a Hardware/Software Approach*. Morgan Kaufman, 1999.
 - [4] GHARACHORLOO, K. Memory consistency models for shared-memory multiprocessors. Tech. Rep. CSL-TR-95-685, Computer Systems Laboratory, Departments of Electrical Engineering and Computer Science, Stanford University, Stanford, CA, December 1995. Available: <http://www.hpl.hp.com/techreports/Compaq-DEC/WRL-95-9.pdf> [Viewed: October 11, 2004].
 - [5] HENNESSY, J. L., AND PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. Morgan Kaufman, 1995.
 - [6] IBM MICROELECTRONICS AND MOTOROLA. *PowerPC Microprocessor Family: The Programming Environments*, 1994.
 - [7] INTEL CORPORATION. *Intel Itanium Architecture Software Developer's Manual Volume 3: Instruction Set Reference*, 2002.
 - [8] INTEL CORPORATION. *Intel Itanium Architecture Software Developer's Manual Volume 3: System Architecture*, 2002.
 - [9] INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 2B: Instruction Set Reference, N-Z*, 2004. Available: <ftp://download.intel.com/design/Pentium4/manuals/25366714.pdf> [Viewed: February 16, 2005].
 - [10] INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 3: System Programming Guide*, 2004. Available: <ftp://download.intel.com/design/Pentium4/manuals/25366814.pdf> [Viewed: February 16, 2005].
 - [11] INTEL CORPORATION. *Intel 64 Architecture Memory Ordering White Paper*, 2007. Available: <http://developer.intel.com/products/processor/manuals/318147.pdf> [Viewed: September 7, 2007].
 - [12] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developers Manual, Volume 3A: System Programming Guide, Part 1*, 2009. Available: <http://download.intel.com/design/processor/manuals/253668.pdf> [Viewed: September 7, 2007].
 - [13] INTERNATIONAL BUSINESS MACHINES CORPORATION. *z/Architecture principles of operation*. Available: <http://publibz.boulder.ibm.com/epubs/pdf/dz9zr003.pdf> [Viewed: February 16, 2005], May 2004.
 - [14] KANE, G. *PA-RISC 2.0 Architecture*. Hewlett-Packard Professional Books, 1996.
 - [15] LYONS, M., SILHA, E., AND HAY, B. PowerPC storage model and AIX programming. Available: <http://www-106.ibm.com/developerworks/eserver/articles/powerpc.html> [Viewed: January 31, 2005], August 2002.
 - [16] MCKENNEY, P. E. Memory ordering in modern microprocessors, part I. *Linux Journal* 1, 136 (August 2005), 52-57. Available: <http://www.linuxjournal.com/article/8211> <http://www.rdrop.com/users/paulmck/scalability/paper/ordering.2007.09.19a.pdf> [Viewed November 30, 2007].
 - [17] MCKENNEY, P. E. Memory ordering in modern microprocessors, part II. *Linux Journal* 1, 137 (September 2005), 78-82. Available: <http://www.linuxjournal.com/article/8212> <http://www.rdrop.com/users/paulmck/scalability/paper/ordering.2007.09.19a.pdf> [Viewed November 30, 2007].
 - [18] MCKENNEY, P. E., AND SILVERA, R. Example power implementation for c/c++ memory

chitecture Programmer's Manual Volumes 1-5, 2002.

【2】ADVANCED MICRO DEVICES.

AMD x86-64 Architecture Programmer's Manual Volume 2: System Programming, 2007.

【3】CULLER, D. E., SINGH, J. P., AND GUPTA, A. *Parallel Computer Architecture: a Hardware/Software Approach*. 劳伦斯·埃里森出版社, 1999.

Morgan Kaufmann 出版社的中文译名通常为“劳伦斯·埃里森出版社”，因为 Morgan Kaufmann 是由 Lawrence Halliday Ellison 创立的。如果需要保留原英文出版社名称，可以改为“劳伦斯·埃里森出版社”。计算机系统工程与计算机科学系，斯坦福大学，1999年12月。可访问：
<http://www.hpl.hp.com/techreports/Compaq-DEC/WRL-95-9.pdf> [引用日期：2004年10月11日]。

【5】HENNESSY, J. L., AND PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 施密特出版社, 1995.

Morgan Kaufmann 出版社的中文译名通常为“施密特出版社”。如果需要保留英文原文，可以改为：[5]

【6】HENNESSY, J. L., AND PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. IBM Microelectronics and Motorola, 1995.

【7】INTEL CORPORATION.

Intel Itanium Architecture Software Developer's Manual Volume 3: Instruction Set Reference, 2002.

【8】INTEL CORPORATION. *Intel Itanium Architecture Software Developer's Manual Volume 3: System Architecture*, 2002.

【9】INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 2B: Instruction Set Reference, N-Z*, 2004. 可在 <ftp://download.intel.com/design/Pentium4/manuals/25366714.pdf> 查阅 [查阅日期：2005年2月16日]。

【10】INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 3: System Programming Guide*, 2004. 获取地址：
<ftp://download.intel.com/design/>

Pentium4/manuals/25366814.pdf [查看日期：2005年2月16日]

【37】Intel 64 Architec-

ture Memory Ordering White Paper, 2007年。

网址：<http://developer.intel.com/products/processor/manuals/318147.pdf> [引用日期：2007年9月7日]。

【12】INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developers Manual, Volume 3A: System Programming Guide, Part 1, 2* 009. 可在 <http://download.intel.com/design/processor/manuals/253668.pdf> 查阅 [查阅日期：2007年9月7日]。

【参考文献】【13】IBM. *z/Architecture 操作原理*. 可在 <http://publibz.boulder.ibm.com/epubs/pdf/dz9zr003.pdf> 查阅 [引用日期：2005年2月16日]。2004年5月。

【14】KANE, G. *PA-RISC 2.0 Architecture*. 惠普专业图书, 1996.

PowerPC存储模型与AIX编程。引用日期：2005年1月31日。出版日期：2002年8月。

现代微处理器中的内存顺序，第一部分。IEEE Spectrum, 136 (2005年8月), 52-57. 可在 <https://www.spectrum.ieee.org/> [2007年11月30日访问] 查阅。

现代微处理器中的内存排序，第二部分。计算机架构新闻快报, 137 (2005年9月), 78-82页。可访问：
<http://www.linuxdevcenter.com/pub/a/linux/2005/09/01/memory-ordering-part2.html> [引用日期：2007年11月30日]

示例电源实现用于c/c内存[v72][v73]

model. Available: <http://www.rdrop.com/users/paulmck/scalability/paper/N2745r.2009.02.27a.html> [Viewed: April 5, 2009], February 2009.

- [19] SITES, R. L., AND WITEK, R. T. *Alpha AXP Architecture*, second ed. Digital Press, 1995.
- [20] SPARC INTERNATIONAL. *The SPARC Architecture Manual*, 1994.

模型版本: <http://www.rdrop.com/users/paulmck/scalability/paper/N2745r.2009.02.27a.html> [查看日期: 2009年4月5日]
, 发布于2009年2月。

[19] SITES, R. L., AND WITEK, R. T. *Alpha AXP Architecture*, 第2版. 数字出版社, 1995.

【20】 SPARC INTERNATIONAL.
The SPARC Architecture Manual, 1994.